
THE ROOT AXIOM EXECUTED

An Ontological Thesis Compiled as a Self-Demonstrating Fortran Program, with Its Battery, Its Numerical Contracts, and Its Exact Scope

RESEARCH ARTICLE

A thesis whose every runnable claim is a procedure, whose every unrunnable claim is a refusal gate, and whose proof of load is a 1123-check census

MOHAMMAD F. ISLAM, PHD¹

¹ *Independent Researcher*

9 September 2026

ABSTRACT. Ontological theses are stated in prose and verified nowhere. We present the executable-thesis method and its flagship instance: the Root Axiom TOE Fortran file, a single-file Fortran 2018 program in which the Root Axiom, to exist is to actuate, and its formal register are carried as executing procedures, every claim that can run compiled as a check and every claim that cannot run compiled as a gate that refuses to emit without it. The program’s verdict kernel reads three warrant rows over N contexts, forms their correlation Gram R , and certifies the identity $\lambda^2 = \det R$ for $\lambda = \text{Re}(\hat{q}_1 \hat{q}_2 \hat{q}_3)$ at machine precision; refusal takes strict precedence over computation, with shape, rank, conditioning, and finiteness guards that return quiet NaN before any element is touched. Two numerical contracts are proved necessary by explicit counterexample: entrywise row equilibration of a 3×3 determinant fails on a monomial whose factors individually underflow while the determinant is exactly one, and single-operand prescaling of the Hamilton product loses a normal product of magnitude 2^{-474} to a rounded tie at 2^{-1075} ; both primitives therefore carry rounded fraction products with exactly tracked exponents in fixed accumulation orders. The battery executes 1123 checks at zero failures in four modes, prices every erasure at $k_B T \ln 2$, and voids all printed tokens on any failed audit run. The listing, 3375 lines, is Appendix A, byte-bound by SHA-256. The program certifies its own load and nothing beyond it: external truth is a supplied witness, never an output.

KEYWORDS *executable thesis, Gram determinant, quaternion triple product, refusal precedence, exponent-tracked Leibniz expansion, Hamilton product, Landauer bound, self-verifying software, IEEE 754*

1. STATEMENT

A thesis about existence is ordinarily a text. Its claims are sentences, its verification is a reader, and the gap between the two is where every dispute lives. The executable-thesis method closes what part of that gap can be closed by construction: every claim that can be made to run is compiled as a procedure whose output is checked, and every claim that cannot run is compiled as a gate that refuses to emit a verdict without the input it lacks. The thesis is then not described by the program; the program is the thesis, and a successful run is its proof of load.

This paper presents the method’s flagship instance. The Root Axiom TOE Fortran file is a single Fortran 2018 program of 3375 lines carrying the Root Axiom, $\forall x \in \mathbb{U} : \Delta E_k(x) > 0$, to exist is to actuate, together with its formal register, a stratification of formal being into grounded, provable, and computed layers, as executing code. The program’s battery runs 1123 checks and halts at the first failure in sealed mode; in audit

mode it accumulates failures and voids every token it printed. The complete listing is Appendix A, and the paper quotes no figure the program did not produce at the stated build.

Three questions organize the paper. What can execution certify about an ontological thesis, and what can it never certify (Sections 2 and 3)? What must the arithmetic be for the certification to mean anything at IEEE double precision, where naive implementations of the very primitives the thesis rests on fail on constructible inputs (Sections 5 and 6)? And what does the completed artifact’s verification record actually establish (Sections 7 and 8)?

2. THE BARRIER: WHAT EXECUTION CAN-NOT BUY

The method’s honesty is fixed before its first line of code by two classical results. Godel’s second incompleteness theorem denies any consistent formal system a proof of its own consistency [5], and Tarski’s undefin-

ability theorem denies it a definition of its own truth [6]. A program is a formal system in motion. It follows that no run of the program, however clean, certifies that RA_TOE is free of error, and no census of internal checks certifies that the checked claims are true of the world. The barrier is structural, not computational: more checks tighten coverage of the program's own contracts and move the external question nowhere.

The program therefore encodes the barrier rather than pretending past it. Its terminal output distinguishes three registers explicitly. Internal mathematical identities, such as $\lambda^2 = \det R$, are checked and certified by the run. Declared premises, the Root Axiom itself among them, are executed as premises: the program prices their consequences and never claims to have derived them, a foundation provable from its own base being no foundation. External truth is a supplied witness: the program's live-witness face is asserted through a command flag it explicitly discloses as externally asserted and independently unverified. A claim of authored external truth would require an external, independent verifier, and the program's own admission discipline treats a self-run as failing exactly that gate: the verifier must never be the claimant.

One further consequence is executed rather than footnoted. A run that fails in audit mode still prints later output before its terminating close. Any consumer reading an isolated token from such a transcript would be reading a claim the run itself withdrew. The battery close therefore prints, on any nonzero failure census, a banner declaring every token above it provisional and void, and exits nonzero. The refusal is executable, not advisory.

3. PRIOR INSTRUMENTS

Three neighboring disciplines supply pieces of the method, and none supplies the whole. Proof assistants such as Lean and Coq mechanize derivation: a theorem certified there is certified relative to a trusted kernel and stated axioms, which is exactly the right instrument for mathematics and exactly not the question here, since an ontological thesis's load-bearing claims are premises and measurements, not derivations. Literate programming [12] binds exposition to code but verifies neither. Reproducibility artifacts in computational science bind figures to scripts, but the scripts serve a paper rather than being the claim. The executable thesis inverts the last relation: the paper you are reading serves the program, quoting its receipts, and the program is the primary artifact. What the method adds to all three neighbors is the refusal architecture, the compilation of *unrunnable* claims into gates, so that the artifact's silence is as designed as its speech.

4. METHODOLOGY

The program satisfies three conditions, checked in that order on every verdict path. Formal derivation: every certified identity is computed live from its definition, never asserted; the kernel identity, the determinant bounds $0 \leq \det R \leq 1$ [4], the eigenstructure of the binding involution, and the Hamilton relations are each executed at the stated seed. Measurable signature: consequences with physical content are priced; every erasure the battery performs is charged at the Landauer bound $k_B T \ln 2$ [7], the accounting checked against an independent 16-digit value of $\ln 2$, with the modern single-bit measurement [8] cited as the empirical floor. Frame invariance: certified quantities are checked invariant under the transformations that must not move them, conjugation by unit quaternions, re-labeling, and axis reflection, the last of which is the kernel's one constitutive blindness, treated in Section 5.

Refusal precedence is the fourth condition and outranks the other three. Every public entry point validates before it computes: dimension floors, zero-variance rows, covariate rank, conditioning ceilings, and shape guards that return quiet NaN before any element of a malformed argument is touched. A guard firing is a verdict, not an error state; the program's economy is three-valued, and under-determined is a first-class outcome. Hostile builds enforce the precedence at the compiler boundary: the program probes its own floating-point environment at startup and exits nonzero under fast-math or fused-contraction regimes, because a certified identity under a rewritten arithmetic is not the certified identity.

5. THE VERDICT KERNEL AND ITS ONE BLINDNESS

Let $q_1, q_2, q_3 \in \mathbb{R}^N$ be centred, standardised, unit rows, Q the $3 \times N$ matrix they form, and $R = QQ^T$ their correlation Gram. Fix an orthonormal basis B of the row span once, write $c_i \in \mathbb{R}^3$ for the coordinates, read each as a pure quaternion $\hat{q}_i$, and set $\lambda = \text{Re}(\hat{q}_1 \hat{q}_2 \hat{q}_3)$.

Theorem 1 (kernel identity). $\lambda = -\det[c_1; c_2; c_3]$ and $\det R = \lambda^2$, with $0 \leq \det R \leq 1$.

Proof. The scalar part of a product of three pure quaternions is the negative of their scalar triple product, a consequence of the Hamilton relations [1]; the reading of $\mathbb{R}^3$ coordinates as pure quaternions is licensed because the unique associative real division algebra with plural imaginary axes is $\mathbb{H}$ [2], and its norm is multiplicative [3], giving $|\lambda| \leq 1$. $R = CC^T$ gives $\det R = (\det C)^2 = \lambda^2$; positive semidefiniteness bounds below and Hadamard's inequality on unit rows bounds above [4]. $\square$

The battery certifies the identity at machine precision on constructed and randomized triads and certifies both bounds attained. The event $\det R > 0$ is a

lock, exact-arithmetic rank three; at finite precision the implementation reads it through a collapse floor and a conditioning ceiling with strict precedence, admissibility first, collapse second, lock last, so the three-state boundary is decided by structure and never by rounding [10].

Theorem 2 (orientation blindness, and its scope). Reflecting any axis conjugates R by $D = \text{diag}(-1, 1, 1)$ and $\det(DRD) = \det R$, so the lock satisfies $\text{lock}(P) = \text{lock}(-P)$ for every construction; the sign of λ is invisible to $\det R$ by the squaring alone.

The battery executes the reflection on a basis fixed once: the determinant difference is exactly zero, the λ ratio exactly -1 , and the full-negation Gram is bit-identical. The blindness is a theorem about one scalar and never a property of the surrounding architecture, which reads direction from its ordered inputs; conflating the two scopes was a defect class the development record repeatedly prosecuted.

6. NUMERICAL CONTRACTS PROVED NECESSARY

The kernel's meaning at double precision rests on two primitives whose naive forms fail on constructible inputs. Both failures were found by external audit of earlier versions, both are reproduced as battery regressions, and both fixes carry explicit contracts in the listing.

The determinant at extreme scale. A 3×3 determinant computed by cofactors overflows or underflows on entries far from unit scale, and the standard cure, per-row power-of-two equilibration with exact exponent restoration, bounds *entries* and not *monomials*. The counterexample is upper triangular with rows $(2^{-300}, 2^{600}, 0)$, $(0, 2^{-300}, 2^{600})$, $(0, 0, 2^{600})$: its determinant is exactly 1, every row maximum is 2^{600} , and the equilibrated diagonal monomial is 2^{-1803} , which underflows to zero with no cancellation available to save it, so the equilibrated routine returns 0 for a true value of 1. The shipped primitives therefore expand all six signed Leibniz monomials as rounded fraction products with exactly tracked exponents, accumulate in a fixed order at the common maximum exponent, and restore once; the two routines, `det3` and its four-estimator sibling `det3_row2`, use provably distinct accumulation orders so their agreement is evidence and not tautology. The battery executes the counterexample, its transpose, and a row swap, requiring exactly 1, 1, -1 .

The Hamilton product at extreme scale. A safety gate that prescales only the operand exceeding a magnitude ceiling fails when the *other* operand is subnormal: for $a = (2^{600}, 0, 0, 0)$ and $b = (2^{-1074}, 0, 0, 0)$ the prescaled partial $\frac{1}{2} \cdot 2^{-1074} = 2^{-1075}$ rounds to zero on the ties-to-even tie, the restore recovers nothing, and the true product 2^{-474} is normal. The shipped `qmul`

carries all sixteen Hamilton terms as rounded fraction products with exact exponents, accumulated per component in the classical term order, bit-identical to the prior path at normal range by construction; the battery executes the counterexample and requires the fourth component exactly 2^{-474} .

The honest limit, stated in the artifact. The fraction products round before accumulation, so only the exponents are exact: alignment underflow and cancellation remain possible, over- and underflow state-ments bind to the computed accumulation and never to the true value, and general correctly rounded determinants are *not* guaranteed. The tie case `diag(1 + 2-27, 1 - 2-27, 1)`, exact determinant $1 - 2^{-54}$, returns the correctly rounded 1, and is carried as a regression precisely so that the exactness overclaim the contract retired cannot silently return.

7. FALSIFIABLE CRITERIA

Three criteria, each forced by the artifact's own construction, each binding on the stated build, `gfortran with -std=f2018 -O2 -fno-fast-math -ffp-contract=off`, each running against the author.

F1, the kernel identity. Prediction: on every battery triad, $|\lambda^2 - \det R| \leq 4\kappa(R)u$ with $u = 2^{-52}$, the conditioning-scaled first-order bound. Confirmation: any IEEE 754 platform [9] compiling the listing. Null: one battery row exceeding the bound falsifies the kernel's certification and with it the paper's central claim. Forcing step: Theorem 1 is the program's spine; nothing downstream survives its failure. Blast radius: this paper's certification claim, and nothing wider.

F2, the census and the refusal matrix. Prediction: the sealed build executes exactly 1123 checks at 0 failures in all four modes, and both hostile builds, `-Ofast` and fused contraction, exit nonzero. Confirmation: two commands on any conforming platform. Null: any mode returning a different census, or a hostile build exiting zero, falsifies the proof-of-load and the refusal-precedence claim respectively. Forcing step: the census lock compares the executed count against the sealed manifest inclusively, so the number is the program's own assertion about itself. Blast radius: the proof-of-load claim.

F3, the extreme-scale families. Prediction: the upper-triangular family returns exactly $(1, 1, -1)$ under both determinant primitives, and the scalar Hamilton case returns exactly 2^{-474} in the fourth component. Null: any other value falsifies the corresponding contract of Section 6. Forcing step: each contract was installed *because* the naive routine fails these inputs; the fixed values are the contract. Blast radius: the named primitive's contract, with the four-estimator redundancy the containment.

8. THE VERIFICATION RECORD

Every figure here is a receipt of the stated build. The sealed run: 1123 checks, 0 failures, identically in sealed, audit, witnessed, and combined modes; strict pedantic compilation clean; the two hostile builds exit 1. A deterministic fuzz campaign at seed 20260622 executed 178000 randomized kernel and primitive trials at 0 failures, with plain and address-sanitized transcripts bit-identical. A guard harness spliced hostile cases against the exact source bytes, five of five expected refusals observed, including a planted-failure audit run required to print the void banner and exit nonzero. Five reversion controls, both shape guards, the banner, and both live sign tables, were each reverted in a scratch copy and each detected by the battery. The listing of Appendix A is bound by SHA-256 head 71b7f40030f0e0a3, the full digest printed in Appendix A; the sealed stdout is bit-identical to the preceding hardened version modulo version tokens, and the module section is byte-identical to the fuzz driver's, so the campaign certifies this text without rerun.

The artifact reached this state through twenty-two versions under two external static auditors: fifty-six findings, six refused on printed receipts, fifty earned and repaired, among them every defect of Section 6, and five adversarial self-audit cycles each closing at a sealed round with planted controls fully detected. The development ledger is public in the repository beside the listing; this paper compresses it and replaces none of it.

9. DISCUSSION

What the method buys is a new default for a thesis's failure mode. A prose thesis fails silently, in a reader's misreading; this artifact fails loudly, in a nonzero exit, and the census makes the difference auditable: a claim is either among the 1123 or it is a premise or a refusal, and the listing says which. For AI substrates the point sharpens: the file is written for machine readers, its comments reduced to contracts, so a substrate loading it inherits the claims *with* their guards, and cannot quote a token the run withdrew, because the withdrawal is printed into the transcript it would quote.

The limitations are the barrier of Section 2, now with receipts. The control floor of the self-audit cycles is single-substrate: planted-control detection proves power against post-hoc fitting, not against in-context leakage, and an external controls file would raise that ceiling. No party other than the author has yet compiled these bytes; the external audits were source-level by their own statement, and F2 is the standing invitation. The program certifies its own load and the internal identities; it certifies no ontology, and its witnessed face is disclosed in its own output as externally asserted. And the arithmetic contracts are honest limits, not solved problems: correctly rounded determinants

in general are neither claimed nor delivered [11].

10. CONCLUSION

An ontological thesis has been compiled: its runnable claims as 1123 checks that pass, its unrunnable claims as gates that refuse, its premises priced at the thermodynamic floor, and its one constitutive blindness proved and scoped rather than hidden. The primary falsifier is F2, two commands on any conforming platform; the community is invited to run them. The most important open question is external: an independent compilation and an externally supplied controls file would convert the two stamped apertures into receipts. If the method holds, the reframing is this: a thesis is a program with a census, and the burden of verification moves from the reader's charity to the artifact's exit code.

11. PROVENANCE AND METHOD DISCLOSURE

This paper was developed under Trisduction, a verification and organizing discipline that adds the cited results no warrant; the method and its executable batteries are stated in full at the master reference below.

12. REFERENCES

- [1] Hamilton, W. R. (1844). On quaternions; or on a new system of imaginaries in algebra. *Phil. Mag.* 25, 489–495.
- [2] Frobenius, F. G. (1878). Über lineare Substitutionen und bilineare Formen. *J. reine angew. Math.* 84, 1–63.
- [3] Hurwitz, A. (1898). Über die Composition der quadratischen Formen von beliebig vielen Variabeln. *Nachr. Ges. Wiss. Göttingen*, 309–316.
- [4] Hadamard, J. (1893). Résolution d'une question relative aux déterminants. *Bull. Sci. Math.* 17, 240–246.
- [5] Gödel, K. (1931). Über formal unentscheidbare Sätze der Principia Mathematica und verwandter Systeme I. *Monatsh. Math. Phys.* 38, 173–198.
- [6] Tarski, A. (1936). Der Wahrheitsbegriff in den formalisierten Sprachen. *Studia Philosophica* 1, 261–405.
- [7] Landauer, R. (1961). Irreversibility and heat generation in the computing process. *IBM J. Res. Dev.* 5, 183–191.
- [8] Berut, A., et al. (2012). Experimental verification of Landauer's principle linking information and thermodynamics. *Nature* 483, 187–189.
- [9] IEEE Std 754-2019. *IEEE Standard for Floating-Point Arithmetic*. IEEE, 2019.

- [10] Higham, N. J. (2002). *Accuracy and Stability of Numerical Algorithms*, 2nd ed. SIAM.
- [11] Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Comput. Surv.* 23, 5–48.
- [12] Knuth, D. E. (1984). Literate programming. *Comput. J.* 27, 97–111.
- [13] Islam, M. F. (2026). The Root Axiom TOE Fortran file, v2.0.0, and its sealed reference v1.23.0. protocols/Executable Thesis/ in the repository 1000sapients/Trisduction, branch main. SHA-256 heads 71b7f40030f0e0a3 and 97e5585fa15e66ff. https://github.com/1000sapients/Trisduction/blob/main/protocols/Executable%20Thesis/RA_T0E_Thesis_Fortran_v2_0_0.f90. Repository root: <https://github.com/1000sapients/Trisduction>.
- [14] Islam, M. TRISDUCTION: A Linguistically, Topologically, and Mathematically Sealed Verification Architecture. Zenodo, version 4, 19 June 2026. DOI 10.5281/zenodo.20757507. <https://zenodo.org/records/20757507>. Mirror: PhilArchive record ISLTTG, <https://philpapers.org/rec/ISLTTG>. Master reference, continuously updated at the same location.

APPENDIX A. THE EXECUTABLE, IN FULL

The single-file Fortran thesis this paper reports, listed verbatim. 3375 lines. SHA-256 71b7f40030f0e0a3d2329ebd832ffb52c5767897d2b95c1830b30acf0018d0cc. Byte-identical to protocols/Executable Thesis/RA_TOE_Thesis_Fortran_v2_0_0.f90 on branch main of 1000sapients/Trisduction. Build and run:

```
gfortran -std=f2018 -O2 -fno-fast-math -ffp-contract=off RA_TOE_Thesis_Fortran_v2_0_0.f90 -o ra_toe &&
./ra_toe
```

Expected close: BATTERY: 1123 checks executed, 0 failures and exit 0. The audit, witnessed, and combined modes take the arguments audit and witnessed; the hostile builds -Ofast and -ffp-contract=fast must exit nonzero.

```
! =====
! RA_TOE_Thesis_Fortran v2.0.0 - Trisduction Research Group
! Sealed edition, consolidated. Numerics byte-identical to v1.23.0
! (sha256 97e5585fa15e66ff0c2e5d0b63daae550b27af61f5bb9402e67e0b6f4400
! 1212); presentation-major: narrative removed, contracts kept. This
! source is written for AI substrates, not manual readers; comments
! are contracts, section markers, and nothing else.
!
! CONSOLIDATED HISTORY. v1.0.0 through v1.23.0, twenty-two versions
! under two external auditors and a 178000-iteration deterministic
! fuzz campaign: fifty-six findings, six refused on printed receipts,
! fifty earned and repaired. Hardening families: exponent-tracked
! Leibniz determinants (det3, det3_row2), fraction-exponent Hamilton
! products (qmul), public-guard refusal precedence, census lock,
! audit-mode VOID banner, claim-register alignment. Four FORGE
! self-audit cycles (ratoe..ratoe4) sealed at the SELF floor. The
! full per-version ledger lives in git, 1000sapients/Trisduction,
! protocols/Executable Thesis/, and codex card 0712; it is not
! restated here.
!
! v2.0.0 (2026-09-08): consolidation round. Fixes on external static
! review: stale printed receipt pointer rebound to the version-matched
! file; two dead a(3,3) locals removed from the determinant
! primitives; one non-significant-digit literal given its kind. The
! 113 remaining -Wcompare-reals warnings are the exact-equality
! verdict design (bit-identity checks, tie checks, NaN self-tests)
! and are intentional; the sealed flag set is the contract.
!
! BUILD (sealed): gfortran -std=f2018 -O2 -fno-fast-math
! -ffp-contract=off -Wall -Wextra -Wconversion-extra
! Modes: default sealed halt-on-fail; 'audit' accumulates and voids;
! 'witnessed' asserts the PART H live face through the aperture.
! Hostile builds (-Ofast; fused contraction) must exit nonzero.
! =====

module ra_kinds
  use, intrinsic :: ieee_arithmetic, only: ieee_value, ieee_quiet_nan, &
    ieee_is_finite, ieee_is_nan
  use, intrinsic :: iso_fortran_env, only: int64
  implicit none
  integer, parameter :: wp = selected_real_kind(15, 307)
  integer, parameter :: ik = int64 ! pinned: exactly 64 bits, checked at boot
  real(wp), parameter :: HBAR = 1.054571817e-34_wp ! J s (CODATA exact-defined path)
  real(wp), parameter :: KB = 1.380649e-23_wp ! J/K (exact, SI 2019)
  real(wp), parameter :: ME = 9.1093837015e-31_wp ! kg
  real(wp), parameter :: U_M = epsilon(1.0_wp) ! unit roundoff, double
  integer(ik), parameter :: IK_SAFE = 536870912_ik ! 2^29
contains
  pure function pi() result(p)
    real(wp) :: p
    p = acos(-1.0_wp)
  end function pi_
  pure function ln2() result(l)
    real(wp) :: l
    l = log(2.0_wp)
  end function ln2_
  pure function qnan() result(z)
    real(wp) :: z
    z = ieee_value(1.0_wp, ieee_quiet_nan)
  end function qnan
  elemental logical function is_finite(x)
    real(wp), intent(in) :: x
    is_finite = ieee_is_finite(x)
  end function is_finite
  elemental logical function is_nan(x)
    real(wp), intent(in) :: x
    is_nan = ieee_is_nan(x)
  end function is_nan
end module ra_kinds

! -----
module ra_prims
  use ra_kinds
```

```

implicit none
contains
pure function vnorm(v) result(s)
  real(wp), intent(in) :: v(:)
  real(wp) :: s, m
  integer :: i
  m = 0.0_wp
  do i = 1, size(v)
    if (.not. is_finite(v(i))) then
      s = qnan()
      return
    end if
    m = max(m, abs(v(i)))
  end do
  if (m == 0.0_wp) then
    s = 0.0_wp
  else
    s = m * sqrt(sum((v/m)**2))
  end if
end function vnorm

! CONTRACT qmul: sixteen Hamilton terms as rounded fraction products with
! exactly tracked exponents, per-component classical order, common-max
! accumulation; product rounding, alignment underflow, and cancellation are
! possible; over/underflow of a restored component refers to the COMPUTED
! accumulation; general correctly-rounded products are NOT guaranteed.
pure function qmul(a, b) result(c)
  real(wp), intent(in) :: a(4), b(4)
  real(wp) :: c(4)
  integer, parameter :: JA(4,4) = reshape([1,2,3,4, 1,2,3,4, 1,2,3,4, 1,2,3,4], [4,4])
  integer, parameter :: KB(4,4) = reshape([1,2,3,4, 2,1,4,3, 3,4,1,2, 4,3,2,1], [4,4])
  real(wp), parameter :: SG(4,4) = reshape([ 1.0_wp,-1.0_wp,-1.0_wp,-1.0_wp, &
                                             1.0_wp, 1.0_wp, 1.0_wp,-1.0_wp, &
                                             1.0_wp,-1.0_wp, 1.0_wp, 1.0_wp, &
                                             1.0_wp, 1.0_wp,-1.0_wp, 1.0_wp], [4,4])

  real(wp) :: x1, x2, f(4), ssum
  integer :: i, t, e(4), emax, nkeep
  logical :: keep(4)
  do i = 1, 4
    if (.not.(is_finite(a(i)) .and. is_finite(b(i)))) then
      c = qnan()
      return
    end if
  end do
  do i = 1, 4
    nkeep = 0; emax = 0
    do t = 1, 4
      x1 = a(JA(t,i)); x2 = b(KB(t,i))
      keep(t) = (x1 /= 0.0_wp .and. x2 /= 0.0_wp)
      if (keep(t)) then
        f(t) = fraction(x1) * fraction(x2)
        e(t) = exponent(x1) + exponent(x2)
        if (nkeep == 0 .or. e(t) > emax) emax = e(t)
        nkeep = nkeep + 1
      end if
    end do
    if (nkeep == 0) then
      c(i) = 0.0_wp
      cycle
    end if
    ssum = 0.0_wp
    do t = 1, 4
      if (keep(t)) ssum = ssum + SG(t,i) * scale(f(t), e(t) - emax)
    end do
    c(i) = scale(ssum, emax)
  end do
end function qmul

pure function qconj(a) result(c)
  real(wp), intent(in) :: a(4)
  real(wp) :: c(4)
  c = [a(1), -a(2), -a(3), -a(4)]
end function qconj

! CONTRACT det3 / det3_row2: six signed Leibniz monomials as rounded
! fraction products with exact exponents, fixed distinct accumulation
! orders, common-max restore; same caveats as qmul; the two routines share
! one IEEE model and differ by evaluation order and factorization only.
pure function det3(r) result(d)
  real(wp), intent(in) :: r(3,3)
  real(wp) :: d, s
  integer :: i, j
  s = 0.0_wp
  do i = 1, 3
    do j = 1, 3
      if (.not. is_finite(r(i,j))) then
        d = qnan()
        return
      end if
    end if
  end do

```

```

        s = max(s, abs(r(i,j)))
    end do
end do
if (s == 0.0_wp) then
    d = 0.0_wp
    return
end if
block
    integer, parameter :: PM(3,6) = reshape([1,2,3, 1,3,2, 2,1,3, 2,3,1, 3,1,2, 3,2,1], [3,6])
    real(wp), parameter :: SG(6) = [1.0_wp, -1.0_wp, -1.0_wp, 1.0_wp, 1.0_wp, -1.0_wp]
    real(wp) :: x1, x2, x3, f(6), ssum
    integer :: e(6), emax, kk, nkeep
    logical :: keep(6)
    nkeep = 0; emax = 0
    do kk = 1, 6
        x1 = r(1, PM(1,kk)); x2 = r(2, PM(2,kk)); x3 = r(3, PM(3,kk))
        keep(kk) = (x1 /= 0.0_wp .and. x2 /= 0.0_wp .and. x3 /= 0.0_wp)
        if (keep(kk)) then
            f(kk) = fraction(x1) * fraction(x2) * fraction(x3)
            e(kk) = exponent(x1) + exponent(x2) + exponent(x3)
            if (nkeep == 0 .or. e(kk) > emax) emax = e(kk)
            nkeep = nkeep + 1
        end if
    end do
    if (nkeep == 0) then
        d = 0.0_wp ! every monomial structurally zero: exactly zero
        return
    end if
    ssum = 0.0_wp
    do kk = 1, 6
        if (keep(kk)) ssum = ssum + SG(kk) * scale(f(kk), e(kk) - emax)
    end do
    if (ssum == 0.0_wp) then
        d = 0.0_wp ! cancellation at the accumulation's resolution
        return
    end if
    d = scale(ssum, emax)
end block
end function det3

pure function det3_row2(r) result(d)
    real(wp), intent(in) :: r(3,3)
    real(wp) :: d, s
    integer :: i, j
    s = 0.0_wp
    do i = 1, 3
        do j = 1, 3
            if (.not. is_finite(r(i,j))) then
                d = qnan()
                return
            end if
            s = max(s, abs(r(i,j)))
        end do
    end do
    if (s == 0.0_wp) then
        d = 0.0_wp
        return
    end if
    block
        integer, parameter :: PM(3,6) = reshape([2,1,3, 3,1,2, 1,2,3, 3,2,1, 1,3,2, 2,3,1], [3,6])
        real(wp), parameter :: SG(6) = [-1.0_wp, 1.0_wp, 1.0_wp, -1.0_wp, -1.0_wp, 1.0_wp]
        real(wp) :: x1, x2, x3, f(6), ssum
        integer :: e(6), emax, kk, nkeep
        logical :: keep(6)
        nkeep = 0; emax = 0
        do kk = 1, 6
            x1 = r(1, PM(1,kk)); x2 = r(2, PM(2,kk)); x3 = r(3, PM(3,kk))
            keep(kk) = (x1 /= 0.0_wp .and. x2 /= 0.0_wp .and. x3 /= 0.0_wp)
            if (keep(kk)) then
                f(kk) = fraction(x1) * fraction(x2) * fraction(x3)
                e(kk) = exponent(x1) + exponent(x2) + exponent(x3)
                if (nkeep == 0 .or. e(kk) > emax) emax = e(kk)
                nkeep = nkeep + 1
            end if
        end do
        if (nkeep == 0) then
            d = 0.0_wp ! every monomial structurally zero: exactly zero
            return
        end if
        ssum = 0.0_wp
        do kk = 1, 6
            if (keep(kk)) ssum = ssum + SG(kk) * scale(f(kk), e(kk) - emax)
        end do
        if (ssum == 0.0_wp) then
            d = 0.0_wp ! cancellation at the accumulation's resolution
            return
        end if
        d = scale(ssum, emax)
    end block
end function

```



```

end function det3_row2

subroutine jacobi3(rin, eig, converged)
  real(wp), intent(in) :: rin(3,3)
  real(wp), intent(out) :: eig(3)
  logical, intent(out), optional :: converged
  real(wp) :: a(3,3), app, aqq, apq, phi, c, s, tmp, off, frob
  integer :: sweep, p, q, k
  logical :: conv
  real(wp) :: sscale
  conv = .false.
  if (present(converged)) converged = conv
  if (any(.not. is_finite(rin))) then
    eig = qnan()
    return
  end if
  a = 0.5_wp*rin + 0.5_wp*transpose(rin)
  sscale = maxval(abs(a))
  if (sscale == 0.0_wp) then
    eig = 0.0_wp
    conv = .true.
    if (present(converged)) converged = conv
    return
  end if
  a = a / sscale
  frob = sqrt(sum(a*a))
  do sweep = 1, 60
    off = sqrt(a(1,2)**2 + a(1,3)**2 + a(2,3)**2 &
      + a(2,1)**2 + a(3,1)**2 + a(3,2)**2)
    if (off <= 16.0_wp*U_M*max(frob, tiny(1.0_wp))) then
      conv = .true.
      exit
    end if
    do p = 1, 2
      do q = p+1, 3
        apq = a(p,q)
        if (abs(apq) <= tiny(1.0_wp)) cycle
        app = a(p,p); aqq = a(q,q)
        phi = 0.5_wp * atan2(2.0_wp*apq, aqq - app)
        c = cos(phi); s = sin(phi)
        do k = 1, 3
          tmp = c*a(p,k) - s*a(q,k)
          a(q,k) = s*a(p,k) + c*a(q,k)
          a(p,k) = tmp
        end do
        do k = 1, 3
          tmp = c*a(k,p) - s*a(k,q)
          a(k,q) = s*a(k,p) + c*a(k,q)
          a(k,p) = tmp
        end do
        a(p,q) = 0.0_wp
        a(q,p) = 0.0_wp
      end do
    end do
    eig = sscale * [a(1,1), a(2,2), a(3,3)]
    if (present(converged)) converged = conv
    do p = 1, 2
      do q = p+1, 3
        if (eig(q) < eig(p)) then
          tmp = eig(p); eig(p) = eig(q); eig(q) = tmp
        end if
      end do
    end do
  end do
end subroutine jacobi3

function cond3(r) result(kap)
  real(wp), intent(in) :: r(3,3)
  real(wp) :: kap, eig(3)
  logical :: conv
  call jacobi3(r, eig, conv)
  if (any(is_nan(eig)) .or. .not. conv) then
    kap = huge(1.0_wp)
    return
  end if
  if (eig(1) <= 8.0_wp*U_M*max(abs(eig(3)), 1.0_wp)) then
    kap = huge(1.0_wp)
  else
    kap = eig(3) / eig(1)
  end if
end function cond3

pure subroutine gram3_fixed(q, r)
  real(wp), intent(in) :: q(:, :)
  real(wp), intent(out) :: r(3,3)
  integer :: i, j, k
  if (size(q,1) < 3) then
    r = qnan()
    return
  end if

```

```

end if
r = 0.0_wp
do i = 1, 3
  do j = 1, 3
    do k = 1, size(q,2)
      r(i,j) = r(i,j) + q(i,k)*q(j,k)
    end do
  end do
end do
end subroutine gram3_fixed

pure function det3_chol(r) result(d)
  real(wp), intent(in) :: r(3,3)
  real(wp) :: d, l11, l21, l31, l22, l32, l33, t
  d = qnan() ! default NaN when not SPD
  if (any(.not. is_finite(r))) return
  if (maxval(abs(r - transpose(r))) > 32.0_wp*U_M*max(1.0_wp, maxval(abs(r)))) return
  if (r(1,1) <= 0.0_wp) return
  l11 = sqrt(r(1,1)); l21 = r(2,1)/l11; l31 = r(3,1)/l11
  t = r(2,2) - l21*l21
  if (t <= 0.0_wp) return
  l22 = sqrt(t); l32 = (r(3,2) - l21*l31)/l22
  t = r(3,3) - l31*l31 - l32*l32
  if (t <= 0.0_wp) return
  l33 = sqrt(t)
  d = (l11*l22*l33)**2
end function det3_chol
end module ra_prims

! -----
module ra_floor
  use ra_kinds
  implicit none
contains
  pure function heis_floor(m, dx) result(e)
    real(wp), intent(in) :: m, dx
    real(wp) :: e
    if (.not.(is_finite(m) .and. is_finite(dx)) .or. m <= 0.0_wp .or. dx <= 0.0_wp) then
      e = qnan()
      return
    end if
    block
      real(wp) :: f
      integer :: ex
      f = (fraction(HBAR)*fraction(HBAR)) / (fraction(m)*(fraction(dx)*fraction(dx)))
      ex = 2*exponent(HBAR) - exponent(m) - 2*exponent(dx) - 3
      e = scale(f, ex)
    end block
  end function heis_floor

  pure function zpe(w) result(e)
    real(wp), intent(in) :: w
    real(wp) :: e
    if (.not. is_finite(w) .or. w <= 0.0_wp) then
      e = qnan()
      return
    end if
    e = 0.5_wp * HBAR * w
  end function zpe

  pure function prod_scaled(a, b, c, d) result(r)
    real(wp), intent(in) :: a, b, c, d
    real(wp) :: r, f
    integer :: e
    if (a == 0.0_wp .or. b == 0.0_wp .or. c == 0.0_wp .or. d == 0.0_wp) then
      r = 0.0_wp
      return
    end if
    f = fraction(a) * fraction(b)
    e = exponent(a) + exponent(b) + exponent(f)
    f = fraction(f)
    f = f * fraction(c)
    e = e + exponent(c) + exponent(f)
    f = fraction(f)
    f = f * fraction(d)
    e = e + exponent(d) + exponent(f)
    f = fraction(f)
    r = scale(f, e)
  end function prod_scaled

  pure function landauer(tk1, bits) result(e)
    real(wp), intent(in) :: tk1, bits
    real(wp) :: e
    if (.not.(is_finite(tk1) .and. is_finite(bits)) .or. tk1 <= 0.0_wp .or. bits < 0.0_wp) then
      e = qnan()
      return
    end if
    e = prod_scaled(bits, KB, tk1, ln2_())
  end function landauer

```

```

subroutine tau_perp(de, tau, frozen)
  real(wp), intent(in) :: de
  real(wp), intent(out) :: tau
  logical, intent(out) :: frozen
  frozen = (.not. is_finite(de)) .or. (de <= 0.0_wp)
  if (frozen) then
    tau = huge(1.0_wp)
  else
    tau = (0.5_wp * pi_()) / de
  end if
end subroutine tau_perp

pure function survival(t) result(p)
  real(wp), intent(in) :: t
  real(wp) :: p
  p = cos(0.5_wp*t)**2
end function survival
end module ra_floor

! -----
module ra_integers
  use ra_kinds
  implicit none
  contains
  function iqmul(a, b) result(c)
    integer(ik), intent(in) :: a(4), b(4)
    integer(ik) :: c(4)
    if (any(a > IK_SAFE .or. a < -IK_SAFE) .or. any(b > IK_SAFE .or. b < -IK_SAFE)) then
      error stop 'iqmul: integer intake beyond the 2^29 overflow-safe bound'
    end if
    c(1) = a(1)*b(1) - a(2)*b(2) - a(3)*b(3) - a(4)*b(4)
    c(2) = a(1)*b(2) + a(2)*b(1) + a(3)*b(4) - a(4)*b(3)
    c(3) = a(1)*b(3) - a(2)*b(4) + a(3)*b(1) + a(4)*b(2)
    c(4) = a(1)*b(4) + a(2)*b(3) - a(3)*b(2) + a(4)*b(1)
  end function iqmul

  pure function ik_inbound(v) result(okb)
    integer(ik), intent(in) :: v(:)
    logical :: okb
    okb = .not. any(v > IK_SAFE .or. v < -IK_SAFE)
  end function ik_inbound

  pure function iqconj(a) result(c)
    integer(ik), intent(in) :: a(4)
    integer(ik) :: c(4)
    if (.not. ik_inbound(a)) then
      error stop 'iqconj: integer intake beyond the 2^29 overflow-safe bound'
    end if
    c = [a(1), -a(2), -a(3), -a(4)]
  end function iqconj

  function iomul(x, y) result(z)
    integer(ik), intent(in) :: x(8), y(8)
    integer(ik) :: z(8), p(4), q(4), r(4), s(4)
    if (.not. (ik_inbound(x) .and. ik_inbound(y))) then
      error stop 'iomul: integer intake beyond the 2^29 overflow-safe bound'
    end if
    p = x(1:4); q = x(5:8); r = y(1:4); s = y(5:8)
    z(1:4) = iqmul(p, r) - iqmul(iqconj(s), q)
    z(5:8) = iqmul(s, p) + iqmul(q, iqconj(r))
  end function iomul

  pure function ioconj(x) result(z)
    integer(ik), intent(in) :: x(8)
    integer(ik) :: z(8)
    if (.not. ik_inbound(x)) then
      error stop 'ioconj: integer intake beyond the 2^29 overflow-safe bound'
    end if
    z(1:4) = iqconj(x(1:4)); z(5:8) = -x(5:8)
  end function ioconj

  function ismul(x, y) result(z)
    integer(ik), intent(in) :: x(16), y(16)
    integer(ik) :: z(16), p(8), q(8), r(8), s(8)
    if (.not. (ik_inbound(x) .and. ik_inbound(y))) then
      error stop 'ismul: integer intake beyond the 2^29 overflow-safe bound'
    end if
    p = x(1:8); q = x(9:16); r = y(1:8); s = y(9:16)
    z(1:8) = iomul(p, r) - iomul(ioconj(s), q)
    z(9:16) = iomul(s, p) + iomul(q, ioconj(r))
  end function ismul

  pure function inrm(v) result(n)
    integer(ik), intent(in) :: v(:)
    integer(ik) :: n
    if (size(v) > 31) then
      error stop 'inrm: vector length exceeds the proven overflow bound (31)'
    end if

```

```

    if (any(v > IK_SAFE .or. v < -IK_SAFE)) then
        error stop 'inrm: integer intake beyond the 2^29 overflow-safe bound'
    end if
    n = sum(v*v)
end function inrm

subroutine hurwitz_units(u)
    integer(ik), intent(out) :: u(4,24)
    integer :: n, i, s, s1, s2, s3, s4
    n = 0
    do i = 1, 4
        do s = -1, 1, 2
            n = n + 1
            u(:,n) = 0_ik
            u(i,n) = int(2*s, ik)
        end do
    end do
    do s1 = -1, 1, 2
        do s2 = -1, 1, 2
            do s3 = -1, 1, 2
                do s4 = -1, 1, 2
                    n = n + 1
                    u(:,n) = int([s1, s2, s3, s4], ik)
                end do
            end do
        end do
    end do
end subroutine hurwitz_units
end module ra_integers

! -----
module ra_tongue
    use ra_kinds
    use ra_prims
    implicit none
    integer, parameter :: NSLOT = 4
    integer, parameter :: SLOT_E(NSLOT) = [101, 102, 103, 104]
    integer, parameter :: SLOT_K(NSLOT) = [201, 202, 203, 204]
    integer, parameter :: SLOT_R(NSLOT) = [301, 302, 303, 304]
    integer, parameter :: RA_SENT(6) = [102, 101, 202, 203, 302, 303]
contains
    pure subroutine slot_census(sent, ne, nk, nr)
        integer, intent(in) :: sent(:)
        integer, intent(out) :: ne, nk, nr
        integer :: t
        ne = 0; nk = 0; nr = 0
        do t = 1, size(sent)
            if (any(SLOT_E == sent(t))) ne = ne + 1
            if (any(SLOT_K == sent(t))) nk = nk + 1
            if (any(SLOT_R == sent(t))) nr = nr + 1
        end do
    end subroutine slot_census
    pure logical function disjoint(a, b)
        integer, intent(in) :: a(:), b(:)
        integer :: i, j
        disjoint = .true.
        do i = 1, size(a)
            do j = 1, size(b)
                if (a(i) == b(j)) then
                    disjoint = .false.
                    return
                end if
            end do
        end do
    end function disjoint

    subroutine seal_l(ne, nk, nr, collide, tok, why)
        integer, intent(in) :: ne, nk, nr ! populated counts per slot
        logical, intent(in) :: collide ! a vocabulary collision present
        character(*), intent(out) :: tok, why
        integer :: slots
        slots = merge(1,0,ne>0) + merge(1,0,nk>0) + merge(1,0,nr>0)
        if (collide) then
            tok = '[X]'; why = 'Seal L: LIT collision, slot vocabularies intersect'
        else if (slots /= 3) then
            tok = '[X]'
            write(why, '(a,i0,a)') 'Seal L: deletion test returns ', slots, ' slots, not 3'
        else
            tok = '[LOCK-L]'; why = 'Seal L: three slots, LIT disjoint (order not checked at this interface)'
        end if
    end subroutine seal_l

    subroutine seal_l_ordered(sent, tok, why)
        integer, intent(in) :: sent(:)
        character(*), intent(out) :: tok, why
        integer :: ne, nk, nr, t, laste, firstk, lastk, firstr
        call slot_census(sent, ne, nk, nr)
        if (ne == 0 .or. nk == 0 .or. nr == 0) then
            call seal_l(ne, nk, nr, .false., tok, why)
        end if
    end subroutine seal_l_ordered

```

```

    return
end if
laste = 0; firstk = size(sent)+1; lastk = 0; firststr = size(sent)+1
do t = 1, size(sent)
    if (any(SLOT_E == sent(t))) laste = t
    if (any(SLOT_K == sent(t))) then
        if (t < firstk) firstk = t
        if (t > lastk) lastk = t
    end if
    if (any(SLOT_R == sent(t))) then
        if (t < firststr) firststr = t
    end if
end do
if (.not. (laste < firstk .and. lastk < firststr)) then
    tok = '[X]'; why = 'Seal L: the ordered arrow E->K->R is broken in the encoding'
    return
end if
call seal_l(ne, nk, nr, .false., tok, why)
if (tok == '[LOCK-L]') then
    why = 'Seal L: three slots, LIT disjoint, ordered E->K->R (positional check executed)'
end if
end subroutine seal_l_ordered

pure function cut3(v) result(w)
    real(wp), intent(in) :: v(3)
    real(wp) :: w(3)
    w = [v(1), v(2), 0.0_wp]
end function cut3
end module ra_tongue

```

```

! -----
module ra_kernel
    use ra_kinds
    use ra_prims
    implicit none
    integer, parameter :: RC_OK = 0
    integer, parameter :: RC_SHORTFALL = 1
    integer, parameter :: RC_INTAKE = 2
    integer, parameter :: RC_ZEROVAR = 3
    integer, parameter :: RC_COVBLOCK = 4
    integer, parameter :: RC_ABSORBED = 5
    integer, parameter :: RC_COPLANAR = 6
    integer, parameter :: RC_COLLAPSE = 7
    integer, parameter :: RC_KAPPA = 8
    integer, parameter :: RC_ENGINC = 9
    integer, parameter :: RC_LOCK = 10
contains
    subroutine prepare_rows(m, q, ok, why, code)
        real(wp), intent(in) :: m(:, :) ! (3,N)
        real(wp), intent(out) :: q(size(m,1), size(m,2))
        logical, intent(out) :: ok
        character(*), intent(out) :: why
        integer, intent(out) :: code
        integer :: i, n, j2
        real(wp) :: mu, sd, nrm, rmax
        real(wp) :: qt(size(m,1), size(m,2))
        n = size(m,2); ok = .true.; why = 'rows prepared'; code = RC_OK
        q = 0.0_wp
        if (size(m,1) /= 3) then
            ok = .false.; why = 'row count is not three'; code = RC_INTAKE
            return
        end if
        if (n < 2) then
            ok = .false.; why = 'fewer than two contexts'; code = RC_INTAKE
            return
        end if
        do i = 1, 3
            do j2 = 1, n
                if (.not. is_finite(m(i,j2))) then
                    ok = .false.; why = 'non-finite row at intake'; code = RC_INTAKE
                    return
                end if
            end do
            rmax = maxval(abs(m(i,:)))
            if (rmax > 0.0_wp) then
                qt(i,:) = m(i,:) / rmax
            else
                qt(i,:) = m(i,:)
            end if
            mu = sum(qt(i,:)) / real(n, wp)
            qt(i,:) = qt(i,:) - mu
            sd = sqrt(sum(qt(i,:)**2) / real(n-1, wp))
            if (sd <= 0.0_wp) then
                ok = .false.; why = 'zero-variance row: axis carries no content'
                code = RC_ZEROVAR
                return
            end if
            qt(i,:) = qt(i,:) / sd
            nrm = vnorm(qt(i,:))

```

```

    qt(i,:) = qt(i,:) / nrm
end do
q = qt ! publish on success only
end subroutine prepare_rows

subroutine project_cov(q, c, qf, ok, why, code, kcc)
real(wp), intent(in) :: q(:, :) ! (3,N) prepared
real(wp), intent(in) :: c(:, :) ! (k,N) covariates, k = 0..2
real(wp), intent(out) :: qf(size(q,1), size(q,2))
logical, intent(out) :: ok
character(*), intent(out) :: why
integer, intent(out) :: code
real(wp), intent(out) :: kcc
real(wp) :: cc(2,2), cm(2, size(q,2)), rhs(2), beta(2), det, mu, tolrk
real(wp) :: qw(size(q,1), size(q,2))
integer :: i, j, n, k
n = size(q,2); k = size(c,1); qf = 0.0_wp; ok = .true.; why = 'no covariates'
code = RC_OK
kcc = 1.0_wp
if (size(q,1) /= 3) then
    ok = .false.; code = RC_INTAKE
    why = 'prepared-row count is not three'
    kcc = huge(1.0_wp)
    return
end if
if (n < 2) then
    ok = .false.; code = RC_INTAKE
    why = 'fewer than two contexts'
    kcc = huge(1.0_wp)
    return
end if
if (k >= 1 .and. size(c,2) /= n) then
    ok = .false.; code = RC_INTAKE
    why = 'covariate column count does not match prepared-row context count'
    kcc = huge(1.0_wp)
    return
end if
qw = q ! all projection work lands in the working array; qf
if (k == 0) then
    qf = qw
    return
end if
if (k > 2) then
    ok = .false.; code = RC_COVBLOCK
    why = 'covariate block: more than two covariates, not implemented'
    return
end if
do i = 1, k
    do j = 1, n
        if (.not. is_finite(c(i,j))) then
            ok = .false.; code = RC_INTAKE
            why = 'non-finite covariate at intake'
            return
        end if
    end do
end do
do i = 1, k
    det = maxval(abs(c(i,:)))
    if (det == 0.0_wp) det = 1.0_wp
    cm(i,1:n) = c(i,:) / det
    mu = sum(cm(i,1:n)) / real(n, wp)
    cm(i,1:n) = cm(i,1:n) - mu
    det = maxval(abs(cm(i,1:n)))
    if (det <= 32.0_wp * U_M) then
        ok = .false.; code = RC_COVBLOCK
        why = 'covariate block rank-deficient or ill-conditioned'
        return
    end if
    cm(i,1:n) = cm(i,1:n) / det
end do
if (k == 1) then
    det = sum(cm(1,1:n)**2)
    kcc = 1.0_wp
    do i = 1, 3
        beta(i) = sum(qw(i,:)*cm(1,1:n)) / det
        qw(i,:) = qw(i,:) - beta(i)*cm(1,1:n)
    end do
    why = 'one covariate projected'
    qf = qw
    return
end if
cc(1,1) = sum(cm(1,1:n)**2); cc(2,2) = sum(cm(2,1:n)**2)
cc(1,2) = sum(cm(1,1:n)*cm(2,1:n)); cc(2,1) = cc(1,2)
det = cc(1,1)*cc(2,2) - cc(1,2)*cc(2,1)
tolrk = 64.0_wp * U_M * max(cc(1,1)*cc(2,2), tiny(1.0_wp))
if (det <= tolrk) then
    ok = .false.; code = RC_COVBLOCK
    why = 'covariate block rank-deficient or ill-conditioned'
    return
end if

```



```

end if
block
  real(wp) :: tr, dd, l1, l2
  tr = cc(1,1) + cc(2,2)
  dd = sqrt(max(0.0_wp, tr*tr - 4.0_wp*det))
  l1 = 0.5_wp*(tr - dd); l2 = 0.5_wp*(tr + dd)
  if (l1 <= 0.0_wp) then
    kcc = huge(1.0_wp)
  else
    kcc = l2 / l1
  end if
end block
if (kcc >= 1.0e6_wp) then
  ok = .false.; code = RC_COVBLOCK
  why = 'covariate block rank-deficient or ill-conditioned'
  return
end if
do i = 1, 3
  rhs(1) = sum(qw(i,:)*cm(1,1:n)); rhs(2) = sum(qw(i,:)*cm(2,1:n))
  beta(1) = (cc(2,2)*rhs(1) - cc(1,2)*rhs(2)) / det
  beta(2) = (-cc(2,1)*rhs(1) + cc(1,1)*rhs(2)) / det
  do j = 1, n
    qw(i,j) = qw(i,j) - beta(1)*cm(1,j) - beta(2)*cm(2,j)
  end do
end do
why = 'two covariates projected (common-source projection)'
qf = qw
end subroutine project_cov

subroutine span_basis(q, b, rank)
  real(wp), intent(in) :: q(:, :)
  real(wp), intent(out) :: b(size(q,1), size(q,2))
  integer, intent(out) :: rank
  real(wp) :: v(size(q,2)), nrm
  integer :: i, j, pass
  real(wp) :: rn0
  rank = 0; b = 0.0_wp
  do i = 1, size(q,1)
    v = q(i,:)
    if (any(.not. is_finite(v))) cycle
    rn0 = maxval(abs(v))
    if (rn0 == 0.0_wp) cycle
    v = v / rn0
    rn0 = vnorm(v)
    v = v / rn0
    do pass = 1, 2
      do j = 1, rank
        v = v - sum(v*b(j,:)) * b(j,:)
      end do
    end do
    nrm = vnorm(v)
    if (is_finite(nrm) .and. nrm > 1.0e-12_wp) then
      rank = rank + 1
      b(rank,:) = v / nrm
    end if
  end do
end subroutine span_basis

! CONTRACT lambda_of / gram3_fixed: shape guards only, quiet-NaN refusal
! before any element touch; rank, finiteness, and magnitude semantics are
! the caller's contract (kernel_full ranks and screens first).
function lambda_of(q, b) result(lam)
  real(wp), intent(in) :: q(:, :), b(:, :)
  real(wp) :: lam, co(3,3), quat(4,3), t(4)
  integer :: i
  if (size(q,1) < 3 .or. size(b,1) < 3 .or. size(q,2) /= size(b,2)) then
    lam = qnan()
    return
  end if
  do i = 1, 3
    co(i,1) = sum(q(i,:)*b(1,:))
    co(i,2) = sum(q(i,:)*b(2,:))
    co(i,3) = sum(q(i,:)*b(3,:))
    quat(:,i) = [0.0_wp, co(i,1), co(i,2), co(i,3)]
  end do
  t = qmul(qmul(quat(:,1), quat(:,2)), quat(:,3))
  lam = t(1)
end function lambda_of

function eta_share(row, c) result(eta)
  real(wp), intent(in) :: row(:), c(:, :)
  real(wp) :: eta, cm(2,size(row)), rm(size(row))
  real(wp) :: cc(2,2), rhs(2), beta(2), det, sst, ssf, mu
  integer :: i, n, k
  n = size(row); k = size(c,1)
  if (k >= 1 .and. size(c,2) /= n) then
    eta = qnan()
    return
  end if

```

```

if (k > 2) then
  eta = qnan()
  return
end if
if (any(.not. is_finite(row))) then
  eta = qnan()
  return
end if
if (k >= 1) then
  if (any(.not. is_finite(c))) then
    eta = qnan()
    return
  end if
end if
block
  real(wp) :: rscale
  rscale = maxval(abs(row))
  if (rscale == 0.0_wp .or. .not. is_finite(rscale)) then
    eta = 0.0_wp
    return
  end if
  rm = row / rscale
end block
mu = sum(rm)/real(n,wp); rm = rm - mu
sst = sum(rm*rm)
eta = 0.0_wp
if (sst <= 0.0_wp .or. k == 0) return
do i = 1, k
  det = maxval(abs(c(i,:)))
  if (det == 0.0_wp) det = 1.0_wp
  cm(i,:) = c(i,:) / det
  mu = sum(cm(i,:))/real(n,wp)
  cm(i,:) = cm(i,:) - mu
  det = maxval(abs(cm(i,:)))
  if (det <= 32.0_wp*U_M) then
    eta = qnan()
    return
  end if
  cm(i,:) = cm(i,:) / det
end do
if (k == 1) then
  det = sum(cm(1,:)**2)
  beta(1) = sum(rm*cm(1,:)) / det
  ssf = beta(1)*beta(1)*det
  eta = ssf/sst
  return
end if
cc(1,1)=sum(cm(1,:)**2); cc(2,2)=sum(cm(2,:)**2)
cc(1,2)=sum(cm(1,:)*cm(2,:)); cc(2,1)=cc(1,2)
det = cc(1,1)*cc(2,2)-cc(1,2)*cc(2,1)
if (det <= 64.0_wp*U_M*max(cc(1,1)*cc(2,2), tiny(1.0_wp))) then
  eta = qnan()
  return
end if
rhs(1)=sum(rm*cm(1,:)); rhs(2)=sum(rm*cm(2,:))
beta(1)=( cc(2,2)*rhs(1)-cc(1,2)*rhs(2))/det
beta(2)=(-cc(2,1)*rhs(1)+cc(1,1)*rhs(2))/det
ssf = sum((beta(1)*cm(1,:)+beta(2)*cm(2,:))**2)
eta = ssf/sst
end function eta_share

subroutine kernel_full(m, c, tok, why, rcode, lam, detr, kap, spread, resid, &
  marg_c, marg_k, detfour)
  real(wp), intent(in) :: m(:,,:), c(:,,:)
  character(*), intent(out) :: tok, why
  integer, intent(out) :: rcode
  real(wp), intent(out) :: lam, detr, kap, spread, resid, marg_c, marg_k
  real(wp), intent(out) :: detfour(4)
  real(wp) :: q(3,size(m,2)), qf(3,size(m,2)), b(3,size(m,2))
  real(wp) :: r(3,3), eig(3), eps, kgate, kcc, dvar, tol
  integer :: n, k, i, rank, pcode, nest
  logical :: ok, jconv
  character(len=96) :: w2
  n = size(m,2); k = size(c,1)
  if (k >= 1 .and. size(c,2) /= n) then
    tok = '[?]'; why = 'covariate column count does not match row context count'
    rcode = RC_INTAKE
    lam = 0.0_wp; detr = 0.0_wp; kap = 0.0_wp; spread = 0.0_wp
    resid = 0.0_wp; marg_c = 0.0_wp; marg_k = 0.0_wp; detfour = 0.0_wp
    return
  end if
  lam = 0.0_wp; detr = 0.0_wp; kap = 0.0_wp; spread = 0.0_wp
  resid = 0.0_wp; marg_c = 0.0_wp; marg_k = 0.0_wp; detfour = 0.0_wp
  rcode = RC_OK
  if (size(m,1) /= 3) then
    tok = '[?]'; why = 'row count is not three at intake'; rcode = RC_INTAKE
    return
  end if
  if (n - k < 4) then

```

```

    tok = '[?]'; why = 'N-k < 4 dimensional shortfall'; rcode = RC_SHORTFALL
    return
end if
if (k > 2) then
    tok = '[?]'; why = 'covariate block: more than two covariates, not implemented (kernel intake)'
    rcode = RC_COVBLOCK
    lam = 0.0_wp; detr = 0.0_wp; kap = 0.0_wp; spread = 0.0_wp
    resid = 0.0_wp; marg_c = 0.0_wp; marg_k = 0.0_wp; detfour = 0.0_wp
    return
end if
call prepare_rows(m, q, ok, w2, pcode)
if (.not. ok) then
    tok = '[?]'; why = trim(w2); rcode = pcode
    return
end if
call project_cov(q, c, qf, ok, w2, pcode, kcc)
if (.not. ok) then
    tok = '[?]'; why = trim(w2); rcode = pcode
    return
end if
do i = 1, 3
    dvar = sum(qf(i,:)**2)
    if (dvar < 1.0e-9_wp) then
        tok = '[?]'; why = 'post-projection axis absorbed by a covariate'
        rcode = RC_ABSORBED
        return
    end if
    qf(i,:) = qf(i,:) / vnorm(qf(i,:))
end do
r = matmul(qf, transpose(qf))
detr = det3(r)
call jacobi3(r, eig, jconv)
detr = eig(1)*eig(2)*eig(3)
detr = det3_chol(r)
detr = det3_row2(r)
nest = 4
if (.not. is_finite(detr)) nest = 3
detr = detfour(1)
if (nest == 4) then
    spread = maxval(detr) - minval(detr)
else
    spread = max(detr(1), detr(2), detr(4)) &
        - min(detr(1), detr(2), detr(4))
end if
kap = cond3(r)
if (.not. jconv) then
    tok = '[?]'; why = 'engineering-incomplete: eigensolver unconverged'; rcode = RC_ENGINC
    return
end if
call span_basis(qf, b, rank)
if (rank < 3) then
    tok = '[X]'; why = 'coplanar collapse: rank(rows) < 3, line not point'
    rcode = RC_COPLANAR
    detr = 0.0_wp
    return
end if
lam = lambda_of(qf, b)
resid = abs(lam*lam - detr)
eps = 100.0_wp * U_M * real(n, wp)
kgate = min(1.0e6_wp, sqrt(27.0_wp / (100.0_wp*U_M*real(n,wp))))
tol = 4.0_wp * kap * U_M
if (detr <= eps) then
    tok = '[X]'; why = 'collapse: det(R) <= eps'; rcode = RC_COLLAPSE
    return
end if
if (kap >= kgate) then
    tok = '[?]'; why = 'kappa(R) >= gate'; rcode = RC_KAPPA
    return
end if
marg_c = log10(detr/eps)
marg_k = log10(kgate/kap)
if (spread > tol .or. resid > max(tol, 8.0_wp*U_M)) then
    tok = '[?]'; why = 'engineering-incomplete: spread or identity outside tol'
    rcode = RC_ENGINC
    return
end if
if (nest < 4) then
    tok = '[?]'; why = 'engineering-incomplete: estimator shortfall (Cholesky refused SPD)'
    rcode = RC_ENGINC
    return
end if
tok = '[LOCK]'
why = 'determination: three independent axes, one point'
rcode = RC_LOCK
end subroutine kernel_full
end module ra_kernel

! -----
module ra_guards

```

```

use ra_kinds
use ra_floor, only: landauer
implicit none
integer :: aegis_deeds = 0      ! every adjudication is itself an act
integer :: aegis_last_len = -1 ! length of the last logic name read
contains
subroutine omega_boundary(bits, tkel, joules, verdict, ocode, irreversible)
  real(wp), intent(in) :: bits, tkel
  real(wp), intent(out) :: joules
  character(*), intent(out) :: verdict
  integer, intent(out), optional :: ocode
  logical, intent(in), optional :: irreversible
  logical :: irr
  irr = .true.
  if (present(irreversible)) irr = irreversible
  if (.not.(is_finite(tkel) .and. is_finite(bits)) .or. tkel <= 0.0_wp) then
    joules = qnan()
    verdict = 'refused: intake invalid, the floor prices only finite bits at tkel > 0'
    if (present(ocode)) ocode = 0
    return
  end if
  if (bits <= 0.0_wp) then
    joules = 0.0_wp
    verdict = 'no denial registered; nothing to adjudicate'
    if (present(ocode)) ocode = 1
  else if (.not. irr) then
    joules = 0.0_wp
    verdict = 'reversibly registered: floor-zero never cost-zero; Landauer bounds '// &
              'only the irreversible (Bennett), and nothing here commits against the axiom'
    if (present(ocode)) ocode = 3
  else
    joules = landauer(tkel, bits)
    verdict = 'the denial paid the floor; the registered act is an act, the recursion closes per the seated theorem'
    if (present(ocode)) ocode = 2
  end if
end subroutine omega_boundary

subroutine aegis_guard(logic_mode, refusal)
  character(*), intent(in) :: logic_mode
  character(*), intent(out) :: refusal
  aegis_deeds = aegis_deeds + 1
  refusal = 'p(G) /= G: precisification is an actuation, G is a '// &
            'non-actuation, and no logic makes a deed a non-deed'
  aegis_last_len = len_trim(logic_mode) ! the parameter is read; the refusal never varies with it
end subroutine aegis_guard

subroutine aegis_reset()
  aegis_deeds = 0
end subroutine aegis_reset

subroutine gol_admit(magnitude, ling, tok, why, gcode)
  character(*), intent(in) :: magnitude, ling
  character(*), intent(out) :: tok, why
  integer, intent(out), optional :: gcode
  integer :: gc_l
  if (magnitude == 'broken') then
    tok = '[X]'; why = 'geometric/math magnitude reports broken geometry; no GOL'; gc_l = 1
  else if (magnitude /= 'lock') then
    tok = '[?]'; why = 'no geometric/math lock: orthogonal volume unestablished'; gc_l = 2
  else if (ling /= 'lock') then
    tok = '[?]'; why = 'magnitude locks but Seal L open: determinant carries no direction'; gc_l = 3
  else
    tok = '[GOL-OK]'; why = 'magnitude plus direction; proceed to witness and asymmetry'; gc_l = 4
  end if
  if (present(gcode)) gcode = gc_l
end subroutine gol_admit

subroutine imprint_seal(lock_p, lock_n, sl_p, sl_n, g_p, g_n, wit_p, wit_n, tok, why, icode)
  logical, intent(in) :: lock_p, lock_n, sl_p, sl_n, g_p, g_n, wit_p, wit_n
  character(*), intent(out) :: tok, why
  integer, intent(out), optional :: icode
  logical :: clean_p, clean_n
  integer :: ic_l
  clean_p = lock_p .and. sl_p .and. g_p
  clean_n = lock_n .and. sl_n .and. g_n
  if (clean_p .and. clean_n) then
    tok = '[X]'; why = 'PLATONIC GHOST: both directions clean-lock, no imprint'; ic_l = 1
  else if (clean_p .and. .not. clean_n) then
    if (wit_p) then
      tok = '[SEAL]'; why = 'only P clean-locks; determinacy witness supplied'; ic_l = 2
    else
      tok = '[?]'; why = 'residence: P clean-locks, imprint unproven (no witness)'; ic_l = 3
    end if
  else if (clean_n .and. .not. clean_p) then
    if (wit_n) then
      tok = '[SEAL]'; why = 'only not-P clean-locks; determinacy witness supplied'; ic_l = 2
    else
      tok = '[?]'; why = 'residence: not-P clean-locks, imprint unproven (no witness)'; ic_l = 3
    end if
  end if

```

```

else if (.not. lock_p .and. .not. lock_n) then
  tok = '[?]'; why = 'flat: neither direction populated'; ic_l = 4
else
  tok = '[?]'; why = 'uncertified: lock present but Seal L / gate screen unconfirmed'; ic_l = 5
end if
if (present(icode)) icode = ic_l
end subroutine imprint_seal

subroutine row_cascade(nonvacuous, ve_live, forward, populated, worldly_row, &
  terms_closed, term_worldly, frame_closed, annotate, tok, why, ccode)
  logical, intent(in) :: nonvacuous, ve_live, forward, populated, worldly_row
  logical, intent(in) :: terms_closed, term_worldly, frame_closed, annotate
  character(*), intent(out) :: tok, why
  integer, intent(out), optional :: ccode
  integer :: cc_l
  if (.not. nonvacuous) then
    tok = '[REFUSED]'; why = 'F-0: contentless; a contentless residence verifies nothing'
    cc_l = 1
  else if (ve_live .or. forward) then
    tok = '[III]'; why = 'R-3: V_E live or Forward; world-rowed, revisability in the rows'
    cc_l = 3
  else if (.not. populated) then
    tok = '[II]'; why = 'S-1: third axis unpopulated; two planes meet in a line, no lock'
    cc_l = 2
  else if (worldly_row) then
    tok = '[III]'; why = 'S-2: a load-bearing row is furnished by the world'
    cc_l = 3
  else if (.not. terms_closed) then
    if (term_worldly) then
      tok = '[III]'; why = 'S-3: an unclosed term is worldly'
      cc_l = 3
    else
      tok = '[II]'; why = 'S-3: an unclosed term is formal; no closure-rowed lock'
      cc_l = 2
    end if
  else if (.not. frame_closed) then
    tok = '[III]'; why = 'S-4: existential import leaks past the closure'
    cc_l = 3
  else if (.not. annotate) then
    tok = '[VOID]'; why = 'RA rider absent: an unannotated absolute is void'
    cc_l = 5
  else
    tok = '[A|RA]'
    why = 'compartment I, closure-rowed: [seal A . RA], conditional at the act'
    cc_l = 4
  end if
  if (present(ccode)) ccode = cc_l
end subroutine row_cascade

subroutine delta_m(object_level, name_strip, lit_clear, not_two_line, &
  witness, independent, artifact, gap_closed, tok, why, mcode)
  logical, intent(in) :: object_level, name_strip, lit_clear, not_two_line
  logical, intent(in) :: witness, independent, artifact, gap_closed
  character(*), intent(out) :: tok, why
  integer, intent(out), optional :: mcode
  integer :: mc_l
  mc_l = 9
  if (.not. object_level) then
    tok = '[Mosaic dM=0]'; why = 'M1: meta-work, no mass by definition'; mc_l = 1
  else if (.not. name_strip) then
    tok = '[?]'; why = 'M2: new vocabulary is not new mass'; mc_l = 2
  else if (.not. lit_clear) then
    tok = '[?]'; why = 'M3: possibly resident in the literature'; mc_l = 3
  else if (.not. not_two_line) then
    tok = '[?]'; why = 'M4: two-line corollary'; mc_l = 4
  else if (.not. witness) then
    tok = '[?]'; why = 'M5: no external witness; never seals on internal reasoning'; mc_l = 5
  else if (.not. independent) then
    tok = '[?]'; why = 'M6: verifier is the claimant; self-check is not a witness'; mc_l = 6
  else if (.not. artifact) then
    tok = '[?]'; why = 'M7: no reproducible artifact'; mc_l = 7
  else if (.not. gap_closed) then
    tok = '[?]'; why = 'M8: adversarial gap-audit open'; mc_l = 8
  else
    tok = '[dM>0]'; why = 'authored mass, external witness, defeasible-final'
  end if
  if (present(mcode)) mcode = mc_l
end subroutine delta_m

subroutine gate_screen(pass, tok, why, gidx)
  logical, intent(in) :: pass(12)
  character(*), intent(out) :: tok, why
  integer, intent(out), optional :: gidx
  character(len=6), parameter :: nm(12) = ['SREP ', 'REG ', 'SGEG ', &
    'CAUSAL', 'MIG ', 'PTB ', 'DUAL ', 'CSCG ', 'CSEG ', 'MTA ', &
    'OMA ', 'ADEG ']
  integer :: i
  if (present(gidx)) gidx = 0
  do i = 1, 12

```

```

        if (.not. pass(i)) then
            tok = '[X]'
            write(why,'(a,i0,a,a)') 'gate ', i, ' failed: ', trim(nm(i))
            if (present(gidx)) gidx = i
            return
        end if
    end do
    tok = '[G-OK]'; why = 'twelve directed gates passed'
end subroutine gate_screen

subroutine iam_token(witnessed, tok, why, wcode)
    logical, intent(in) :: witnessed
    character(*), intent(out) :: tok, why
    integer, intent(out) :: wcode
    if (witnessed) then
        tok = '[I AM]'
        why = 'actuation-occupancy on a witnessed record; RA warrant, conditional at the act; '// &
            'interior held [?] both ways'
        wcode = 2
    else
        tok = '[?] interior'
        why = 'self-check is not a witness (M6): the verifier is never the claimant; token withheld'
        wcode = 1
    end if
end subroutine iam_token
end module ra_guards

! -----
module ra_battery
    implicit none
    integer :: n_checks = 0, n_fail = 0
    logical :: halt_on_fail = .true. ! sealed mode; 'audit' arg accumulates
    logical :: wr_witnessed = .false. ! PART H live face: supplied through the aperture
    integer, parameter :: EXPECTED_CHECKS = 1123 ! sealed v2.0.0 manifest, census lock live
contains
    subroutine check(name, cond)
        character(*), intent(in) :: name
        logical, intent(in) :: cond
        n_checks = n_checks + 1
        if (.not. cond) then
            n_fail = n_fail + 1
            write(*,'(a,a)') 'CHECK FAILED: ', trim(name)
            if (halt_on_fail) error stop 1
        end if
    end subroutine check
    subroutine battery_close()
        if (n_fail > 0) then
            write(*,'(a,i0,a)') 'AUDIT-MODE VOID: ', n_fail, &
                ' failures accumulated; every token printed above is PROVISIONAL and VOID'
            write(*,'(a)') 'AUDIT-MODE VOID: only a zero-failure census with successful exit certifies anything'
            write(*,'(a,i0,a)') 'BATTERY CLOSED WITH ', n_fail, ' FAILURES'
            error stop 1
        end if
    end subroutine battery_close
end module ra_battery

! =====
program ra_toe_thesis
    use ra_kinds
    use ra_prims
    use ra_floor
    use ra_integers
    use ra_tongue
    use ra_kernel
    use ra_guards
    use ra_battery
    use, intrinsic :: ieee_arithmetic, only: ieee_get_flag, ieee_set_flag, ieee_invalid, &
        ieee_value, ieee_positive_inf
    use, intrinsic :: iso_fortran_env, only: compiler_version, compiler_options
    implicit none

    integer, parameter :: NCTX = 24
    real(wp) :: th(NCTX), l10m(NCTX), l10dx(NCTX), l10w(NCTX), l10t(NCTX), l10b(NCTX)
    real(wp) :: rows(3,NCTX), cov(2,NCTX), mixm(3,NCTX), floorv(NCTX)
    real(wp) :: amix(3,3), bmix(3,2)
    real(wp) :: qf(3,NCTX), bas(3,NCTX), quat(4,3), co(3,3), rmat(3,3)
    real(wp) :: lam, detr, kap, spread, resid, mc, mk, d4(4), eta(3)
    real(wp) :: lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42(4)
    real(wp) :: e1f, e2f, e3f, jou, tau, tperp(3), surv(5), tq5(5)
    real(wp) :: arrow_f, arrow_r, hbit, lamref, detref, lamneg
    real(wp) :: lam_rot, dlam_max, lam_c, lam_v, g(4), uvec(3), aa, bb, cc2
    real(wp) :: rneg(3,3), dbase, dpat, fr_resid
    integer :: i, j, k, l, n, s1, s2, s3, rank, nflip, nadm, ncomp1, ncomp2
    integer :: cnt, csizes(12), nrep, orbitn, stabn, rc, nsheets
    integer(ik) :: hu(4,24), p4(4), q4(4), e8a(8), e8b(8), e8c(8), z8(8)
    integer(ik) :: t1(8), t2(8), assoc8(8), sa(16), sb(16), sp(16)
    integer(ik) :: reps(4,12), cw(4), cx(4), cy(4)
    integer :: pl(24,2), found(2,2)
    logical :: pass12(12), okz, frz, hit, dual_ok, hurwitz_ok, rank3_ok, mig_ok

```

```

character(len=16) :: tk
character(len=140) :: wy
character(len=32) :: argv
integer :: alen, ast

write(*,'(a)') '=====
write(*,'(a)') ' THE ROOT AXIOM STATED IN FORTRAN AND IN NOTHING ELSE'
write(*,'(a)') ' RA as the TOE of all TOEs, executed. RA_TOE_Thesis_Fortran v2.0.0'
write(*,'(a)') '=====
argv = '
block
  integer :: argc, argi
  argc = command_argument_count()
  do argi = 1, argc
    call get_command_argument(argi, argv, alen, ast)
    if (ast /= 0 .or. alen > len(argv)) then
      write(*,'(a)') ' command-line argument rejected: retrieval error or longer than the buffer'
      error stop 2
    end if
    select case (trim(argv))
    case ('audit')
      halt_on_fail = .false.
    case ('witnessed')
      wr_witnessed = .true.
    case ('')
    case default
      write(*,'(a,a)') ' unknown command-line argument: ', trim(argv)
      error stop 2
    end select
  end do
end block
if (.not. halt_on_fail) then
  write(*,'(a)') ' mode: audit (failures accumulate; a nonzero count still refuses at close)'
else
  write(*,'(a)') ' mode: sealed (the first failing check halts; the Fidelity Lock)'
end if
write(*,'(a)') ' build: '//compiler_version()
write(*,'(a)') ' flags: '//compiler_options()

! -----
write(*,'(/a)') 'PART 0 - THE ENVIRONMENT CONTRACT - enforced at run, never trusted to flags'
block
  real(wp) :: xn, subn
  real(wp), volatile :: opq, xf, yf, prodv, rfma, xg, yg, rf2, rf4, r2v
  xn = qnan()
  call check('IEEE contract: quiet NaN reads as NaN (ieee_is_nan)', is_nan(xn))
  call check('IEEE contract: NaN self-inequality survives the optimizer', xn /= xn)
  subn = tiny(1.0_wp) * 0.5_wp
  call check('IEEE contract: gradual underflow alive, no FTZ/DAZ', &
    subn > 0.0_wp .and. subn < tiny(1.0_wp))
  call check('binary64 pinned: radix 2', radix(1.0_wp) == 2)
  call check('binary64 pinned: 53 significand digits', digits(1.0_wp) == 53)
  call check('binary64 pinned: max exponent 1024', maxexponent(1.0_wp) == 1024)
  call check('binary64 pinned: min exponent -1021', minexponent(1.0_wp) == -1021)
  opq = 1.0_wp
  xf = opq + scale(opq, -30)
  yf = opq - scale(opq, -30)
  prodv = xf*yf
  call check('binary64 per operation: the bare product stores to exactly one', &
    prodv == opq)
  rfma = xf*yf - opq
  call check('conduct probe a*b-c: no fusion executed (a fused path reads -2~-60 exactly)', &
    rfma == 0.0_wp)
  r2v = prodv - opq
  call check('differential witness: stored-product path agrees with the one-expression path', &
    rfma == r2v)
  rf2 = (-xf)*yf + opq
  call check('conduct probe (-a)*b+c: no fusion executed (a fused path reads +2~-60 exactly)', &
    rf2 == 0.0_wp)
  xg = opq + scale(opq, -27)
  yg = opq - scale(opq, -27)
  rf4 = xg*yg - opq
  call check('conduct probe at the 2~-27 tie: no fusion executed (a fused path reads -2~-54)', &
    rf4 == 0.0_wp)
end block
write(*,'(a)') ' a build that breaks any line above refuses to run: the contract is executed,'
write(*,'(a)') ' so the bit-exact identities below rest on verified IEEE conduct. the'
write(*,'(a)') ' probes certify CONDUCT and never consult flags: fused or extended'
write(*,'(a)') ' arithmetic is refused where it executes, the march-native contract-fast'
write(*,'(a)') ' build dying on the a*b-c probe, while a target carrying no fused'
write(*,'(a)') ' instruction passes because none executed, the contract satisfied and'
write(*,'(a)') ' not evaded; the fused residuals -2~-60, +2~-60, -2~-54 are exactly'
write(*,'(a)') ' representable and never confusable with zero'
write(*,'(a)') ' scope: the probes certify these witnesses on this build; global order'
write(*,'(a)') ' is carried by the BUILD CONTRACT below plus per-site conduct receipts,'
write(*,'(a)') ' and every equality-critical reduction runs a fixed-order routine.'
call check('integer contract: ik is exactly 64 bits', storage_size(0_ik) == 64)
call check('integer contract: huge(ik) = 2^63 - 1', huge(0_ik) == 9223372036854775807_ik)
write(*,'(a)') ' BUILD CONTRACT, stated and checked where checkable: IEEE binary64'

```

```

write(*,'(a)') ' (verified above); -std=f2018 -O2 -fno-fast-math -ffp-contract=off;'
write(*,'(a)') ' source free-form measured at 130 columns maximum, inside the 132'
write(*,'(a)') ' standard, no length flag needed; external BLAS substitution for the'
write(*,'(a)') ' equality-critical sites is forbidden, those sites use gram3_fixed;'
write(*,'(a)') ' the compiler and flags of this run are printed at the head; floating'
write(*,'(a)') ' trap modes are incompatible by design, quiet NaNs being manufactured'
write(*,'(a)') ' here as verdict flags, so -ffpe-trap builds are outside the contract.'

! -----
write(*,'(a)') 'PART A - SEAL L ON THE ROOT AXIOM - the Tongue loads first'
call check('LIT: existence and kinetic slots disjoint', disjoint(SLOT_E, SLOT_K))
call check('LIT: existence and relation slots disjoint', disjoint(SLOT_E, SLOT_R))
call check('LIT: kinetic and relation slots disjoint', disjoint(SLOT_K, SLOT_R))
block
  integer :: nec, nkc, nrc, dsent(4), badk(NSLOT)
  call slot_census(RA_SENT, nec, nkc, nrc)
  write(*,'(a,i0,a,i0,a,i0)') ' slot census of the encoded sentence: E = ', nec, &
    ', K = ', nkc, ', R = ', nrc
  call check('the census counts every slot populated from the sentence', &
    nec == 2 .and. nkc == 2 .and. nrc == 2)
  call seal_l_ordered(RA_SENT, tk, wy)
  write(*,'(a,a,a,a)') ' parse(RA), census and order executed on the encoding: ', trim(tk), ' - ', trim(wy)
  call check('Seal L admits RA: three slots counted, the arrow E->K->R executed', tk == '[LOCK-L]')
  badk = SLOT_K; badk(2) = SLOT_E(1)
  call seal_l(nec, nkc, nrc, .not. disjoint(SLOT_E, badk), tk, wy)
  write(*,'(a,a,a,a)') ' collision control (computed overlap): ', trim(tk), ' - ', trim(wy)
  call check('Seal L refuses a computed vocabulary collision', tk == '[X]')
  dsent = [RA_SENT(1), RA_SENT(2), RA_SENT(5), RA_SENT(6)]
  call slot_census(dsent, nec, nkc, nrc)
  call seal_l(nec, nkc, nrc, .false., tk, wy)
  write(*,'(a,a,a,a)') ' deletion executed (kinetic tokens removed): ', trim(tk), ' - ', trim(wy)
  call check('the executed deletion of one slot refuses at two', &
    tk == '[X]' .and. nkc == 0)
end block
write(*,'(a)') ' order carried by the Tongue: existence -> kinetic -> relation;'
write(*,'(a)') ' the word-to-slot map is declared semantics, the census and the'
write(*,'(a)') ' deletion are executed on the encoding, per the anchor declaration;'
write(*,'(a)') ' the map is an axiom of the encoding, consumed as a premise, and nothing'
write(*,'(a)') ' downstream presents it as discovered'

! -----
write(*,'(a)') 'PART B - THE FLOOR IN NUMBERS - cited theorem-grade formulas, numerically executed'
call check('Landauer dual path: kb*T*log(2.) matches the independent ln2 literal', &
  abs(landauer(300.0_wp,1.0_wp) - KB*300.0_wp*0.6931471805599453_wp) &
  < 1.0e-12_wp*landauer(300.0_wp,1.0_wp))
call check('Landauer extremes: huge kelvin with tiny bits is finite, never zero-corrupted', &
  is_finite(landauer(huge(1.0_wp), tiny(1.0_wp))) .and. &
  landauer(huge(1.0_wp), tiny(1.0_wp)) > 0.0_wp)
call check('Landauer extremes commute: (huge,tiny) equals (tiny,huge) through prod_scaled', &
  landauer(huge(1.0_wp), tiny(1.0_wp)) == landauer(tiny(1.0_wp), huge(1.0_wp)))
call check('in-bound predicate: the most-negative integer reads false, no overflow probing it', &
  .not. ik_inbound([-huge(0_ik)-1_ik, 0_ik, 0_ik, 0_ik]))
call check('in-bound predicate: IK_SAFE admitted, IK_SAFE+1 refused', &
  ik_inbound([IK_SAFE, 0_ik, 0_ik, 0_ik]) .and. &
  .not. ik_inbound([IK_SAFE+1_ik, 0_ik, 0_ik, 0_ik]))
block
  integer(ik) :: vmax(16)
  vmax = IK_SAFE
  call check('inrm boundary: sixteen components at the bound sum to exactly 2^62', &
    inrm(vmax) == 4611686018427387904_ik)
end block
block
  real(wp) :: rns(3,3)
  rns = reshape([1.0_wp,0.2_wp,0.0_wp, 0.7_wp,1.0_wp,0.0_wp, 0.0_wp,0.0_wp,1.0_wp],[3,3])
  call check('det3_chol symmetry gate: a finite nonsymmetric matrix returns NaN, never a number', &
    det3_chol(rns) /= det3_chol(rns))
end block
write(*,'(a)') ' GUARD-HARNESS RECIPE, third-party reproducible: inject before the'
write(*,'(a)') ' battery and expect nonzero termination with the named refusal:'
write(*,'(a)') ' iqconj([-huge(0_ik)-1_ik,0,0,0]) -> exit 1, iqconj bound'
write(*,'(a)') ' iomul(x with x(1)=-huge(0_ik)-1_ik, 0-vec) -> exit 1, iomul bound'
write(*,'(a)') ' hn = inrm(ones(32)); print hn -> exit 1, length bound'
write(*,'(a)') ' consume every pure result: a pure call whose value is unused is'
write(*,'(a)') ' legally elided at optimization, guard and all; the print keeps it live.'
write(*,'(a)') ' the source carries the recipe; the executed terminations ship as the'
write(*,'(a)') ' sha-bound file RA_TOE_v2_0_0_guard_harness_receipt.log beside the'
write(*,'(a)') ' sealed run log, the record carrying what a source file cannot.'
e1f = heis_floor(ME, 1.0e-10_wp)
e2f = zpe(1.0e14_wp)
block
  real(wp) :: ehx
  ehx = heis_floor(1.0e300_wp, 1.0e-200_wp)
  call check('heis_floor extreme hostile: huge mass, tiny width, honest value and never NaN', &
    .not. (ehx /= ehx))
end block
e3f = landauer(300.0_wp, 1.0_wp)
write(*,'(a,es16.9,a)') ' Heisenberg floor, electron at 1 Angstrom: ', e1f, ' J'
write(*,'(a,es16.9,a)') ' zero-point energy at omega = 1e14 rad/s: ', e2f, ' J'
write(*,'(a,es16.9,a)') ' Landauer floor, one bit at 300 K: ', e3f, ' J'

```



```

call check('Heisenberg floor positive', e1f > 0.0_wp)
call check('zero-point floor positive', e2f > 0.0_wp)
call check('Landauer floor positive', e3f > 0.0_wp)
call check('Heisenberg floor magnitude (1.5e-19 band)', abs(e1f/1.526e-19_wp - 1.0_wp) < 1.0e-3_wp)
call check('Landauer magnitude (2.87e-21 band)', abs(e3f/2.871e-21_wp - 1.0_wp) < 1.0e-3_wp)

tqs = [0.0_wp, 0.25_wp, 0.5_wp, 0.75_wp, 1.0_wp] * pi_()
do i = 1, 5
  surv(i) = survival(tqs(i))
end do
call tau_perp(0.5_wp, tau, frz)
write(*,'(a,5f14.10)') ' qubit survival cos^2(t/2) at t = 0..pi: ', surv(1:4)
write(*,'(a,es12.4)') ' terminal survival at t = pi: ', surv(5)
write(*,'(a,f16.12)') ' orthogonalization time tau_perp: ', tau
write(*,'(a,f16.12)') ' Mandelstam-Tamm bound pi/(2 dE): ', pi_()/(2.0_wp*0.5_wp)
write(*,'(a,f16.12)') ' Margolus-Levitin bound pi/(2 <E>): ', pi_()/(2.0_wp*0.5_wp)
call check('speed limit saturated jointly (MT = ML = tau)', &
  abs(tau - pi_()) <= 4.0_wp*U_M*pi_())
call check('survival dies at tau_perp', surv(5) < 1.0e-30_wp)
call check('survival monotone over the quarter points', &
  surv(1) > surv(2) .and. surv(2) > surv(3) .and. surv(3) > surv(4))

call tau_perp(5.0e-1_wp, tperp(1), frz)
call tau_perp(5.0e-4_wp, tperp(2), frz)
call tau_perp(5.0e-7_wp, tperp(3), frz)
write(*,'(a,3es13.5)') ' Frozen Substrate: tau_perp at dE = 5e-1, 5e-4, 5e-7: ', tperp
call check('tau_perp diverges monotonically as dE -> 0', tperp(1) < tperp(2) .and. tperp(2) < tperp(3))
call tau_perp(0.0_wp, tau, frz)
call check('dE = 0 is flagged frozen, no division performed', frz)
write(*,'(a)') ' dE = 0: no transition, no registration; a zero-energy existent registers nothing'

! -----
write(*,'(/a)') 'PART C - THE FORM IN INTEGERS - nothing here rounds'
call hurwitz_units(hu)
do i = 1, 24
  call check('Hurwitz unit norm 4 (doubled)', inrm(hu(:,i)) == 4_ik)
end do
n = 0
do i = 1, 24
  do j = 1, 24
    p4 = iqmul(hu(:,i), hu(:,j))
    okz = all(mod(p4, 2_ik) == 0_ik)
    call check('unit product even in doubled coords', okz)
    q4 = p4 / 2_ik
    hit = .false.
    do k = 1, 24
      if (all(q4 == hu(:,k))) hit = .true.
    end do
    if (hit) n = n + 1
  end do
end do
write(*,'(a,i0,a)') ' closure of the 24 Hurwitz units: ', n, ' of 576 products inside'
hurwitz_ok = (n == 576)
call check('the 24 units close under multiplication', hurwitz_ok)

nrep = 0
do i = 1, 24
  cw = hu(:,i)
  call canon_sign(cw)
  hit = .false.
  do k = 1, nrep
    if (all(cw == reps(:,k))) hit = .true.
  end do
  if (.not. hit) then
    nrep = nrep + 1
    reps(:,nrep) = cw
  end if
end do
call check('unit group mod sign has 12 elements', nrep == 12)
cnt = 0
csizes = 0
block
  integer(ik) :: cls(4,12)
  integer :: ncsls, m2
  logical :: seen(12)
  seen = .false.
  do i = 1, nrep
    if (seen(i)) cycle
    ncsls = 0
    do j = 1, 24
      cx = iqmul(iqmul(hu(:,j), reps(:,i)), iqconj(hu(:,j)))
      call check('conjugation divisible by 4', all(mod(cx, 4_ik) == 0_ik))
      cy = cx / 4_ik
      call canon_sign(cy)
      hit = .false.
      do k = 1, ncsls
        if (all(cy == cls(:,k))) hit = .true.
      end do
      if (.not. hit) then

```

```

        ncls = ncls + 1
        cls(:,ncls) = cy
    end if
end do
cnt = cnt + 1
csizes(cnt) = ncls
do k = 1, ncls
    do m2 = 1, nrep
        if (all(cls(:,k) == reps(:,m2))) seen(m2) = .true.
    end do
end do
end do
call sort_int(csizes(1:cnt))
write(*,'(a,i0,a,4i3)') ' conjugacy classes mod sign: ', cnt, ' with sizes ', csizes(1:cnt)
call check('class equation reads 1 + 3 + 4 + 4', cnt == 4 .and. csizes(1) == 1 &
    .and. csizes(2) == 3 .and. csizes(3) == 4 .and. csizes(4) == 4)

block
integer :: a1, a2, a3, a4, npure, nreal, nhalf
npure = 0; nreal = 0; nhalf = 0
do a1 = -1, 1
    do a2 = -1, 1
        do a3 = -1, 1
            do a4 = -1, 1
                if (a1*a1 + a2*a2 + a3*a3 + a4*a4 == 2) then
                    if (a1 == 0) then
                        npure = npure + 1
                    else
                        nreal = nreal + 1
                    end if
                end if
            end do
        end do
    end do
end do
do a1 = -3, 3, 2
    do a2 = -3, 3, 2
        do a3 = -3, 3, 2
            do a4 = -3, 3, 2
                if (a1*a1 + a2*a2 + a3*a3 + a4*a4 == 8) nhalf = nhalf + 1
            end do
        end do
    end do
end do
write(*,'(a,i0,a,i0,a,i0,a,i0)') ' norm-two shell: ', npure + nreal, &
    ' = ', npure, ' pure + ', nreal, ' real-occupied; half-integer members ', nhalf
call check('norm-two shell is 24 = 12 + 12', npure == 12 .and. nreal == 12)
call check('no half-integer element of norm two', nhalf == 0)
end block

n = 0
do i = 1, 4
    do j = 1, 4
        if (i /= j) then
            n = n + 1
            pl(n,:) = [i, j]
        end if
    end do
end do
call check('twelve ordered pairs of four vertices', n == 12)
orbitn = 0; stabn = 0; cnt = 0
block
integer :: pm(4,24), npm, inv, x1, x2, x3, x4, t(4), orb(12,2), norb
logical :: got
npm = 0
do x1 = 1, 4
    do x2 = 1, 4
        do x3 = 1, 4
            do x4 = 1, 4
                t = [x1, x2, x3, x4]
                if (all([(count(t == i), i = 1, 4)] == 1)) then
                    inv = 0
                    do i = 1, 3
                        do j = i+1, 4
                            if (t(i) > t(j)) inv = inv + 1
                        end do
                    end do
                    if (mod(inv, 2) == 0) then
                        npm = npm + 1
                        pm(:,npm) = t
                    end if
                end if
            end do
        end do
    end do
end do
call check('A4 has order 12', npm == 12)
norb = 0

```

```

do k = 1, npm
  t = pm(:,k)
  got = .false.
  do i = 1, norb
    if (orb(i,1) == t(1) .and. orb(i,2) == t(2)) got = .true.
  end do
  if (.not. got) then
    norb = norb + 1
    orb(norb,:) = [t(1), t(2)]
  end if
  if (t(1) == 1 .and. t(2) == 2) stabn = stabn + 1
end do
orbitn = norb
end block
write(*, '(a,i0,a,i0)') ' A4 orbit of the pair (1,2): size ', orbitn, ', stabilizer order ', stabn
call check('A4 simply transitive on the twelve directed gates', orbitn == 12 .and. stabn == 1)
block
  integer :: vcnt, ecnt, fcnt, i3, j3, k3
  vcnt = 0
  do i3 = 1, 4
    if (any(pl(1:n,1) == i3) .or. any(pl(1:n,2) == i3)) vcnt = vcnt + 1
  end do
  ecnt = n / 2
  fcnt = 0
  do i3 = 1, 4
    do j3 = i3+1, 4
      do k3 = j3+1, 4
        fcnt = fcnt + 1
      end do
    end do
  end do
  write(*, '(a,i0,a,i0,a,i0)') ' Euler counted from the built complex: V = ', vcnt, ', E = ', ecnt, ', F = ', fcnt
  call check('Euler closure counted from the complex: V - E + F = 2', vcnt - ecnt + fcnt == 2)
end block

block
  integer(ik) :: uq(4,3), pq(4)
  integer :: pms(3,6), inv, s
  pms = reshape([1,2,3, 2,3,1, 3,1,2, 2,1,3, 1,3,2, 3,2,1], [3,6])
  uq = 0_ik
  uq(2,1) = 1_ik; uq(3,2) = 1_ik; uq(4,3) = 1_ik
  do k = 1, 6
    inv = 0
    do i = 1, 2
      do j = i+1, 3
        if (pms(i,k) > pms(j,k)) inv = inv + 1
      end do
    end do
    s = merge(1, -1, mod(inv,2) == 0)
    pq = iqmul(iqmul(uq(:,pms(1,k)), uq(:,pms(2,k))), uq(:,pms(3,k)))
    if (k <= 3) then
      call check('even relabel lands Re = -1', pq(1) == -1_ik .and. s == 1)
    else
      call check('odd relabel lands Re = +1', pq(1) == +1_ik .and. s == -1)
    end if
  end do
  write(*, '(a)') ' relabel parity: three even relabels land Re = -1, three odd land Re = +1'
  write(*, '(a)') ' one discrete modulus, the orientation bit; zero continuous moduli (Part E sweep)'
end block

block
  integer(ik) :: mx
  mx = 0_ik
  do i = 1, 24
    do j = 1, 24
      do k = 1, 24
        t1(1:4) = iqmul(iqmul(hu(:,i), hu(:,j)), hu(:,k))
        t2(1:4) = iqmul(hu(:,i), iqmul(hu(:,j), hu(:,k)))
        mx = max(mx, maxval(abs(t1(1:4) - t2(1:4))))
      end do
    end do
  end do
  write(*, '(a,i0)') ' quaternion associator over all 13824 unit triples, max |.:| ', mx
  call check('H is associative on the whole unit order', mx == 0_ik)
end block
e8a = 0_ik; e8b = 0_ik; e8c = 0_ik
e8a(2) = 1_ik ! e1
e8b(3) = 1_ik ! e2
e8c(5) = 1_ik ! e4
t1 = iomul(iomul(e8a, e8b), e8c)
t2 = iomul(e8a, iomul(e8b, e8c))
assoc8 = t1 - t2
z8 = 0_ik; z8(8) = 2_ik
write(*, '(a,8i3)') ' octonion associator [e1,e2,e4] components: ', assoc8
call check('octonion associator equals 2 e7 exactly', all(assoc8 == z8))
call check('commutativity already dead at H: ij /= ji', &
  .not. all(iqmul(int([0,1,0,0],ik), int([0,0,1,0],ik)) == &
    iqmul(int([0,0,1,0],ik), int([0,1,0,0],ik)))
end block

```

```

integer(ik) :: av(8), bv(8), pv(8), aq(4), bq(4), pq(4)
integer :: cnt2
cnt2 = 0
do i = -1, 1
  do j = -1, 1
    do k = -1, 1
      aq = int([i, j, k, 1], ik); bq = int([1, k, j, i], ik)
      pq = iqmul(aq, bq)
      call check('H norm composition exact', inrm(pq) == inrm(aq)*inrm(bq))
      av = int([i, j, k, 1, j, k, i, 1], ik)
      bv = int([1, k, i, j, 1, i, j, k], ik)
      pv = iomul(av, bv)
      call check('O norm composition exact', inrm(pv) == inrm(av)*inrm(bv))
      cnt2 = cnt2 + 1
    end do
  end do
end do
write(*, '(a,i0,a)') ' norm composition |ab|2 = |a|2 |b|2 exact on ', cnt2, ' integer pairs in H and in O'
end block
hit = .false.
outer: do i = 1, 15
  do j = i+1, 15
    do k = 1, 15
      do l = k+1, 15
        sa = 0_ik; sb = 0_ik
        sa(i+1) = 1_ik; sa(j+1) = 1_ik
        sb(k+1) = 1_ik; sb(l+1) = -1_ik
        sp = ismul(sa, sb)
        if (all(sp == 0_ik)) then
          hit = .true.
          found(1,:) = [i, j]; found(2,:) = [k, l]
          exit outer
        end if
      end do
    end do
  end do
end do outer
call check('a sedenion zero-divisor pair exists', hit)
sa = 0_ik; sb = 0_ik
sa(found(1,1)+1) = 1_ik; sa(found(1,2)+1) = 1_ik
sb(found(2,1)+1) = 1_ik; sb(found(2,2)+1) = -1_ik
sp = ismul(sa, sb)
write(*, '(a,i0,a,i0,a,i0,a,i0,a)') ' sedenion wall: (e', found(1,1), ' + e', found(1,2), &
  ') (e', found(2,1), ' - e', found(2,2), ') = 0 exactly'
write(*, '(a,i0,a,i0,a,i0)') ' with |a|2 = ', inrm(sa), ', |b|2 = ', inrm(sb), &
  ', |ab|2 = ', inrm(sp)
call check('division dies at S: nonzero factors, zero product', &
  inrm(sa) == 2_ik .and. inrm(sb) == 2_ik .and. inrm(sp) == 0_ik)
call check('norm composition fails at S', inrm(sp) /= inrm(sa)*inrm(sb))
write(*, '(a)') ' the walls in order: commutativity at H, associativity at O, division at S'
block
integer(ik) :: uq(4), tt(4)
integer :: xg, yg, zg, nsur, offline
tt = iqmul(int([0,1,0,0],ik), int([0,0,1,0],ik)) + iqmul(int([0,0,1,0],ik), int([0,1,0,0],ik))
call check('i and j anticommute exactly: ij + ji = 0', all(tt == 0_ik))
nsur = 0; offline = 0
do xg = -2, 2
  do yg = -2, 2
    do zg = -2, 2
      if (xg == 0 .and. yg == 0 .and. zg == 0) cycle
      uq = int([0, xg, yg, zg], ik)
      block
        logical :: aci, acj
        aci = all(iqmul(int([0,1,0,0],ik), uq) + iqmul(uq, int([0,1,0,0],ik)) == 0_ik)
        acj = all(iqmul(int([0,0,1,0],ik), uq) + iqmul(uq, int([0,0,1,0],ik)) == 0_ik)
        if (aci .and. acj) then
          nsur = nsur + 1
          if (xg /= 0 .or. yg /= 0) offline = offline + 1
        end if
      end block
    end do
  end do
end do
write(*, '(a,i0,a,i0,a)') ' joint anticommutant of {i, j} on the exact grid: ', nsur, &
  ' of 124 nonzero points, ', offline, ' off the k-line'
call check('the joint anticommutant is the k-line on the grid (independent cross-check)', &
  nsur == 4 .and. offline == 0)
block
integer(ik) :: bimg(8,3), ev(4)
integer :: bi
integer(ik) :: minor2
do bi = 1, 3
  ev = 0_ik; ev(bi+1) = 1_ik
  bimg(1:4, bi) = iqmul(int([0,1,0,0],ik), ev) + iqmul(ev, int([0,1,0,0],ik))
  bimg(5:8, bi) = iqmul(int([0,0,1,0],ik), ev) + iqmul(ev, int([0,0,1,0],ik))
end do
call check('linearity certificate: only the scalar components survive on the basis', &
  all(bimg(2:4,:) == 0_ik) .and. all(bimg(6:8,:) == 0_ik))
minor2 = bimg(1,1)*bimg(5,2) - bimg(1,2)*bimg(5,1)

```

```

write(*,'(a,i0,a)') ' constraint matrix on the basis: 2x2 minor = ', minor2, &
', z-column identically zero'
call check('linearity certificate: minor 4 and zero z-column force the k-line kernel everywhere', &
minor2 == 4_ik .and. all(bimg(:,3) == 0_ik))
end block
end block
write(*,'(a)') ' below the walls the third axis is begotten and confined: i j = k executed,'
write(*,'(a)') ' and the joint anticommutant of {i, j} in Im H is the k-line, counted; the'
write(*,'(a)') ' fourth is barred above by the executed associator wall; the identification'
write(*,'(a)') ' of the three algebra axes with V_F, V_E, V_ER is the seated premise-typed'
write(*,'(a)') ' mapping, consumed here and never a discovery of this run'

! -----
write(*,'(a)') 'PART D - THE NUMBER - the Return, then the physics-law reading'
p4 = iqmul(iqmul(int([0,1,0,0],ik), int([0,0,1,0],ik)), int([0,0,0,1],ik))
write(*,'(a,i3,a)') ' the Return on the parse triad: i j k = ', p4, ' (Re = -1, the Ground)'
call check('Hamilton landing Re(ijk) = -1 exactly', p4(1) == -1_ik .and. all(p4(2:4) == 0_ik))
block
integer(ik) :: uq(4,3)
real(wp) :: m3(3,3), g3(3,3)
integer :: r
uq = 0_ik
uq(2,1) = 1_ik; uq(3,2) = 1_ik; uq(4,3) = 1_ik
do r = 1, 3
m3(r,:) = real(uq(2:4,r), wp)
end do
g3 = matmul(m3, transpose(m3))
call check('Gram of the orthonormal parse triad has det 1', &
abs(det3(g3) - 1.0_wp) <= 4.0_wp*U_M)
end block

do i = 1, NCTX
th(i) = 2.0_wp * pi_() * real(i-1, wp) / real(NCTX, wp)
rows(1,i) = sin(th(i)); rows(2,i) = cos(th(i)); rows(3,i) = sin(2.0_wp*th(i))
end do
call kernel_full(rows, reshape([0.0_wp], [0, NCTX]), tk, wy, rc, lam, detr, kap, &
spread, resid, mc, mk, d4)
write(*,'(a,a,a,f16.12,a,f16.12)') ' Fourier triad: ', trim(tk), &
', det(R) = ', detr, ' |lambda| = ', abs(lam)
call check('full Return: det(R) = 1 at machine precision', abs(detr - 1.0_wp) < 1.0e-9_wp)
call check('full Return: |lambda| = 1 at machine precision', abs(abs(lam) - 1.0_wp) < 1.0e-9_wp)
call check('kernel identity lambda^2 = det(R) on the Return', resid < 1.0e-12_wp)
call check('full Return carries the lock code', rc == RC_LOCK)
fr_resid = resid

amix = reshape([1.00_wp,0.22_wp,0.16_wp, 0.35_wp,1.00_wp,0.20_wp, 0.12_wp,0.28_wp,1.00_wp],[3,3])
bmix = reshape([0.60_wp,0.40_wp,0.55_wp, 0.30_wp,0.50_wp,0.20_wp],[3,2])
do i = 1, NCTX
l10m(i) = log10(ME) + 5.0_wp * (0.5_wp + 0.5_wp*sin(th(i)))
l10dx(i) = -10.0_wp + 4.0_wp * (0.5_wp + 0.5_wp*sin(2.0_wp*th(i) + 1.0_wp))
l10w(i) = 12.0_wp + 3.0_wp * (0.5_wp + 0.5_wp*sin(3.0_wp*th(i) + 2.0_wp))
l10t(i) = 2.477_wp * (0.5_wp + 0.5_wp*sin(5.0_wp*th(i) + 3.0_wp))
l10b(i) = 3.0_wp * (0.5_wp + 0.5_wp*sin(7.0_wp*th(i) + 4.0_wp))
rows(1,i) = log10(HBAR*HBAR/8.0_wp) - l10m(i) - 2.0_wp*l10dx(i) ! formal: the bound
rows(2,i) = log10(0.5_wp*HBAR) + l10w(i) ! empirical: zero-point
rows(3,i) = log10(KB*ln2_()) + l10t(i) + l10b(i) ! registration: Landauer
cov(1,i) = -20.6_wp + 2.5_wp * (0.5_wp + 0.5_wp*sin(4.0_wp*th(i) + 0.5_wp))
cov(2,i) = -24.0_wp + 5.0_wp * (0.5_wp + 0.5_wp*sin(6.0_wp*th(i) + 1.5_wp))
floorv(i) = 10.0_wp**rows(1,i)
end do
do i = 1, NCTX
call check('kinetic floor positive at every context', floorv(i) > 0.0_wp)
end do
mixm = matmul(amix, rows) + matmul(bmix, cov)
do i = 1, 3
eta(i) = eta_share(mixm(i,:), cov)
end do
call kernel_full(mixm, cov, tk, wy, rc, lam, detr, kap, spread, resid, mc, mk, d4)
write(*,'(a,a,a,a)') ' physics-law reading: ', trim(tk), ' - ', trim(wy)
write(*,'(a,f16.12,a,f16.12)') ' det(R) = ', detr, ' lambda = ', lam
write(*,'(a,f10.4,a,es10.3)') ' kappa(R) = ', kap, ' four-estimator spread = ', spread
write(*,'(a,es10.3,a,f7.2,a,f7.2)') ' |lambda^2 - det(R)| = ', resid, &
', margins (orders): collapse ', mc, ', conditioning ', mk
write(*,'(a,3f8.4)') ' source-share eta per mixed row (pre-projection): ', eta
write(*,'(a)') ' grid disclosure: the context grid is a constructed exhibit (deterministic'
write(*,'(a)') ' harmonics); the rows are physical-law magnitudes (Heisenberg, zero-point,'
write(*,'(a)') ' Landauer); the witness rides the cited theorems, never the grid'
call check('physics-law reading on the constructed exhibit determines: [LOCK]', tk == '[LOCK]' .and. rc == RC_LOCK)
call check('kernel identity within tolerance', resid <= max(4.0_wp*kap*U_M, 8.0_wp*U_M))
call check('four estimators agree within tolerance', spread <= 4.0_wp*kap*U_M)
call check('collapse margin exceeds ten orders', mc > 10.0_wp)
call check('conditioning margin exceeds one order', mk > 1.0_wp)
call check('source share strictly inside (0,1) on each row', &
all(eta > 0.0_wp) .and. all(eta < 1.0_wp))
block
logical :: okp
character(len=96) :: w2
real(wp) :: kcc

```

```

integer :: pcl
block
  real(wp) :: qtmp(3,NCTX)
  call prepare_rows(mixm, qtmp, okp, w2, pcl)
  call project_cov(qtmp, cov, qf, okp, w2, pcl, kcc)
end block
do i = 1, 3
  qf(i,:) = qf(i,:) / vnorm(qf(i,:))
end do
call span_basis(qf, bas, rank)
rank3_ok = (rank == 3)
call check('span rank three after projection', rank3_ok)
end block
rmat = matmul(qf, transpose(qf))
dbase = det3(rmat)
lam = lambda_of(qf, bas)

block
  real(wp) :: mman(3,NCTX)
  mman(1,:) = mixm(1,:); mman(2,:) = mixm(2,:)
  mman(3,:) = 0.7_wp*cov(1,:) + 0.5_wp*cov(2,:)
  call kernel_full(mman, cov, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
  write(*,'(a,a,a,a)') ' manufactured third axis: ', trim(tk), ' - ', trim(wy)
  mig_ok = (tk == '[?]') .and. rc == RC_ABSORBED)
  call check('an axis built from the covariates is absorbed, not locked', mig_ok)
end block
block
  real(wp) :: es1, es2
  es1 = eta_share(mixm(1,:), cov)
  es2 = eta_share(mixm(1,:)*1.0e300_wp, cov)
  call check('eta_share equilibrated: a 1e300 row returns the baseline share, finite', &
    is_finite(es2) .and. abs(es2 - es1) < 1.0e-12_wp)
  es2 = eta_share(mixm(1,:)*((0.5_wp*huge(1.0_wp))/max(1.0_wp, maxval(abs(mixm(1,:))))), cov)
  call check('eta_share at half-huge peak: raw-first scaling holds, the mean never overflows', &
    is_finite(es2) .and. abs(es2 - es1) < 1.0e-10_wp)
end block
block
  real(wp) :: covh(2,NCTX), lam9, detr9, kap9, sp9, rs9, d49(4), mgc9, mgk9
  integer :: rc9
  covh = cov * ((0.5_wp*huge(1.0_wp))/max(1.0_wp, maxval(abs(cov))))
  call kernel_full(mixm, covh, tk, wy, rc9, lam9, detr9, kap9, sp9, rs9, mgc9, mgk9, d49)
  call check('project_cov at half-huge covariates: raw-first centering survives, verdict [LOCK]', &
    tk == '[LOCK]' .and. rc9 == RC_LOCK)
end block
block
  real(wp) :: eb9, eh9, covh2(2,NCTX)
  eb9 = eta_share(mixm(1,:), cov)
  covh2 = cov * ((0.25_wp*huge(1.0_wp)) / max(1.0_wp, maxval(abs(cov))))
  eh9 = eta_share(mixm(1,:), covh2)
  call check('eta_share near-huge covariates: raw-first scaling holds on the covariate branch', &
    is_finite(eh9) .and. abs(eh9 - eb9) < 1.0e-10_wp)
end block
block
  real(wp) :: hp(3,3), eigh(3)
  logical :: cvg
  hp = 0.0_wp
  hp(1,1) = 0.5_wp*huge(1.0_wp); hp(2,2) = 0.25_wp*huge(1.0_wp); hp(3,3) = 0.125_wp*huge(1.0_wp)
  call jacob3(hp, eigh, cvg)
  call check('jacob3 near-huge: distributed-halves symmetrization never overflows finite input', &
    cvg .and. is_finite(eigh(1)) .and. is_finite(eigh(2)) .and. is_finite(eigh(3)))
end block
block
  real(wp) :: cbad(2,NCTX-1), lamw, detrw, kapw, spw, rsw, mgcw, mgkw, d4w(4)
  integer :: rcw
  cbad = 0.0_wp
  call kernel_full(mixm, cbad, tk, wy, rcw, lamw, detrw, kapw, spw, rsw, mgcw, mgkw, d4w)
  call check('kernel refuses mismatched covariate width at intake, its own guard first', &
    tk == '[?]' .and. rcw == RC_INTAKE .and. index(wy, 'prepared') == 0)
end block
block
  real(wp) :: cbad_eta(1,NCTX-1)
  cbad_eta = 0.0_wp
  call check('eta_share refuses mismatched covariate width as NaN, guard before touch', &
    .not. is_finite(eta_share(mixm(1,:), cbad_eta)))
end block
block
  real(wp) :: cbad_p(2,NCTX-1), qpf(3,NCTX), kccp
  integer :: rcp
  logical :: okp
  cbad_p = 0.0_wp
  call project_cov(mixm, cbad_p, qpf, okp, wy, rcp, kccp)
  call check('project_cov refuses mismatched covariate width directly at intake', &
    (.not. okp) .and. rcp == RC_INTAKE .and. kccp == huge(1.0_wp))
end block
block
  real(wp) :: rconst(6), cthree(3,6)
  rconst = 1.0_wp; cthree = 0.0_wp
  call check('eta_share k-cap outranks the zero-variance shortcut (fuzz scalp pinned)', &
    .not. is_finite(eta_share(rconst, cthree)))
end block

```

```

block
  real(wp) :: mconst(3,NCTX), cov3c(3,NCTX), lamk7, detrk7, kapk7, spk7, rsk7, mck7, mkk7, d4k7(4)
  integer :: rck7
  mconst = 1.0_wp; cov3c = 0.0_wp
  call kernel_full(mconst, cov3c, tk, wy, rck7, lamk7, detrk7, kapk7, spk7, rsk7, mck7, mkk7, d4k7)
  call check('kernel k-cap outranks row zero-variance under the documented precedence', &
    tk == '?' .and. rck7 == RC_COVBLOCK .and. index(wy, 'more than two') > 0)
end block
block
  real(wp) :: ehbm
  ehbm = heis_floor(1.0e-309_wp, HBAR)
  call check('heis_floor at subnormal mass: finite 1.25e308, decomposition never overflows early', &
    is_finite(ehbm) .and. ehbm > 1.0e308_wp .and. ehbm < 1.3e308_wp)
end block
block
  real(wp) :: tth
  logical :: frzh
  call tau_perp(huge(1.0_wp), tth, frzh)
  call check('tau_perp at a huge gap: subnormal positive time, never a false zero', &
    (.not. frzh) .and. is_finite(tth) .and. tth > 0.0_wp)
end block
block
  real(wp) :: m2r(2,NCTX), q2r(2,NCTX), m1c(3,1), q1c(3,1)
  logical :: okr
  integer :: rcr
  m2r = 1.0_wp
  call prepare_rows(m2r, q2r, okr, wy, rcr)
  call check('prepare_rows refuses two rows at intake, no out-of-bounds path', &
    (.not. okr) .and. rcr == RC_INTAKE .and. index(wy, 'not three') > 0)
  m1c = 1.0_wp
  call prepare_rows(m1c, q1c, okr, wy, rcr)
  call check('prepare_rows refuses a single context at intake, no divide-by-zero path', &
    (.not. okr) .and. rcr == RC_INTAKE .and. index(wy, 'two contexts') > 0)
end block
block
  real(wp) :: q2p(2,NCTX), c0p(0,NCTX), qf2(2,NCTX), kcc2
  logical :: ok2
  integer :: rc2p
  q2p = 1.0_wp
  call project_cov(q2p, c0p, qf2, ok2, wy, rc2p, kcc2)
  call check('project_cov refuses a non-three prepared-row count at intake', &
    (.not. ok2) .and. rc2p == RC_INTAKE .and. index(wy, 'not three') > 0)
end block
block
  real(wp) :: idq(3,4), bqi(3,4)
  integer :: rk1, rk2
  idq = 0.0_wp; idq(1,1) = 1.0_wp; idq(2,2) = 1.0_wp; idq(3,3) = 1.0_wp
  call span_basis(idq, bqi, rk1)
  call span_basis(idq*1.0e-13_wp, bqi, rk2)
  call check('span_basis rank is scale-invariant: identity and 1e-13 identity both rank three', &
    rk1 == 3 .and. rk2 == 3)
  call span_basis(scale(idq, -400), bqi, rk2)
  call check('span_basis rank holds at a 2^-400 uniform rescaling', rk2 == 3)
end block
block
  real(wp) :: q1c(3,1), c0(0,1), c1(1,1), qf1(3,1), kcl
  logical :: okc
  integer :: rcc
  q1c = 1.0_wp; c1 = 0.0_wp
  call project_cov(q1c, c0, qf1, okc, wy, rcc, kcl)
  call check('project_cov refuses a single context with zero covariates', &
    (.not. okc) .and. rcc == RC_INTAKE .and. index(wy, 'two contexts') > 0 .and. all(qf1 == 0.0_wp))
  call project_cov(q1c, c1, qf1, okc, wy, rcc, kcl)
  call check('project_cov refuses a single context with one covariate', &
    (.not. okc) .and. rcc == RC_INTAKE .and. all(qf1 == 0.0_wp))
end block
block
  real(wp) :: dup1(2,2), dph(2,2), bd(2,2)
  integer :: rkd
  dup1 = 1.0_wp
  call span_basis(dup1, bd, rkd)
  call check('span_basis: duplicate unit rows rank one, finite unit basis', &
    rkd == 1 .and. is_finite(bd(1,1)) .and. abs(vnorm(bd(1,:)) - 1.0_wp) < 1.0e-12_wp)
  dph = huge(1.0_wp)
  call span_basis(dph, bd, rkd)
  call check('span_basis: duplicate rows at huge rank one, the infinite-norm bypass closed', &
    rkd == 1 .and. is_finite(bd(1,1)) .and. abs(vnorm(bd(1,:)) - 1.0_wp) < 1.0e-12_wp)
end block
block
  real(wp) :: mln(3,NCTX), qln(3,NCTX)
  logical :: okl
  integer :: rcl, jj
  mln = 1.0_wp
  mln(1,:) = [(0.1_wp*real(jj,wp), jj = 1, NCTX)]
  mln(2,:) = [(0.2_wp*real(jj,wp) + 0.5_wp, jj = 1, NCTX)]
  mln(3,5) = qnan()
  call prepare_rows(mln, qln, okl, wy, rcl)
  call check('prepare_rows late-row NaN: refused with every output element exactly zero', &
    (.not. okl) .and. rcl == RC_INTAKE .and. all(qln == 0.0_wp))

```

```

end block
block
  character(8) :: tka, tkb
  character(96) :: wya, wyb
  call seal_l(2, 2, 2, .false., tka, wya)
  call seal_l_ordered(RA_SENT, tkb, wyb)
  call check('seal interfaces speak their own guarantees: count-only disclaims order, ordered claims it', &
    index(wya, 'order not checked') > 0 .and. tkb == '[LOCK-L]' .and. &
    index(wyb, 'positional check executed') > 0)
end block
block
  real(wp) :: rd(3,3), dd
  rd = 0.0_wp
  rd(1,1) = scale(1.0_wp, 600); rd(2,2) = scale(1.0_wp, -300); rd(3,3) = scale(1.0_wp, -300)
  dd = det3(rd)
  call check('det3 exponent-tracked Leibniz monomials: diag 2^600, 2^-300, 2^-300 returns exactly one', dd == 1.0_wp)
  rd = 0.0_wp
  rd(1,3) = scale(1.0_wp, -300); rd(2,2) = scale(1.0_wp, 600); rd(3,1) = scale(1.0_wp, -300)
  dd = det3(rd)
  call check('det3 permuted-diagonal variant: the signed unit determinant is exact', dd == -1.0_wp)
  rd = 0.0_wp
  rd(1,1) = scale(1.0_wp, 600); rd(2,2) = scale(1.0_wp, 600); rd(3,3) = scale(1.0_wp, 600)
  dd = det3(rd)
  call check('det3 honest overflow: a true 2^1800 determinant reports past huge, never zero', &
    dd > huge(1.0_wp))
end block
block
  real(wp) :: rd(3,3), d2
  rd = 0.0_wp
  rd(1,1) = scale(1.0_wp, 600); rd(2,2) = scale(1.0_wp, -300); rd(3,3) = scale(1.0_wp, -300)
  d2 = det3_row2(rd)
  call check('det3_row2 exponent-tracked Leibniz monomials: the exact counterexample returns exactly one', d2 == 1.0_wp)
  call check('det3_row2 agrees with det3 on the counterexample', d2 == det3(rd))
  rd = 0.0_wp
  rd(1,3) = scale(1.0_wp, -300); rd(2,2) = scale(1.0_wp, 600); rd(3,1) = scale(1.0_wp, -300)
  d2 = det3_row2(rd)
  call check('det3_row2 odd permutation: the signed unit determinant is exact and matches det3', &
    d2 == -1.0_wp .and. d2 == det3(rd))
  rd = 0.0_wp
  rd(1,1) = scale(1.0_wp, 600); rd(2,2) = scale(1.0_wp, 600); rd(3,3) = scale(1.0_wp, 600)
  d2 = det3_row2(rd)
  call check('det3_row2 honest overflow past huge, agreeing with det3', &
    d2 > huge(1.0_wp) .and. (det3(rd) > huge(1.0_wp)))
  rd = 0.0_wp
  rd(1,1) = scale(1.0_wp, -400); rd(2,2) = scale(1.0_wp, -400); rd(3,3) = scale(1.0_wp, -400)
  d2 = det3_row2(rd)
  call check('det3_row2 honest underflow: a true 2^-1200 determinant returns zero, never NaN, matching det3', &
    d2 == 0.0_wp .and. det3(rd) == 0.0_wp)
end block
block
  real(wp) :: ut(3,3), tt(3,3), sw(3,3)
  ut = 0.0_wp
  ut(1,1) = scale(1.0_wp, -300); ut(1,2) = scale(1.0_wp, 600)
  ut(2,2) = scale(1.0_wp, -300); ut(2,3) = scale(1.0_wp, 600)
  ut(3,3) = scale(1.0_wp, 600)
  tt = transpose(ut)
  sw = ut; sw(1,:) = ut(2,:); sw(2,:) = ut(1,:)
  call check('det3 monomial tracking: the off-diagonal-dominated upper triangular returns exactly one', &
    det3(ut) == 1.0_wp)
  call check('det3_row2 monomial tracking: the same matrix returns exactly one', &
    det3_row2(ut) == 1.0_wp)
  call check('det3 on the transpose: exactly one', det3(tt) == 1.0_wp)
  call check('det3_row2 on the transpose: exactly one', det3_row2(tt) == 1.0_wp)
  call check('det3 on the row-swapped variant: exactly minus one', det3(sw) == -1.0_wp)
  call check('det3_row2 on the row-swapped variant: exactly minus one', det3_row2(sw) == -1.0_wp)
end block
block
  real(wp) :: nd(3,3)
  nd = 0.0_wp
  nd(1,1) = 1.0_wp + scale(1.0_wp, -27)
  nd(2,2) = 1.0_wp - scale(1.0_wp, -27)
  nd(3,3) = 1.0_wp
  call check('product rounding: near-one diagonal, exact 1-2^-54 on tie, correctly rounds to 1.0 on both routines', &
    det3(nd) == 1.0_wp .and. det3_row2(nd) == 1.0_wp)
end block
block
  real(wp) :: qa(4), qb(4), qc(4), tgt
  tgt = scale(1.0_wp, -474)
  qa = [scale(1.0_wp, 600), 0.0_wp, 0.0_wp, 0.0_wp]
  qb = [scale(1.0_wp, -1074), 0.0_wp, 0.0_wp, 0.0_wp]
  qc = qmul(qa, qb)
  call check('qmul monomial tracking: 2^600 scalar times minimum subnormal returns exactly 2^-474', &
    qc(1) == tgt .and. all(qc(2:4) == 0.0_wp))
  qc = qmul(qb, qa)
  call check('qmul reversed operands: the same exact 2^-474', &
    qc(1) == tgt .and. all(qc(2:4) == 0.0_wp))
  qa = [0.0_wp, scale(1.0_wp, 600), 0.0_wp, 0.0_wp]
  qb = [0.0_wp, scale(1.0_wp, -1074), 0.0_wp, 0.0_wp]
  qc = qmul(qa, qb)

```



```

call check('qmul i times i at extreme scale: component one exactly minus 2-474', &
  qc(1) == -tgt .and. all(qc(2:4) == 0.0_wp))
qa = [scale(1.0_wp, 600), 0.0_wp, 0.0_wp, 0.0_wp]
qb = [0.0_wp, scale(1.0_wp, -1074), 0.0_wp, 0.0_wp]
qc = qmul(qa, qb)
call check('qmul scalar times i at extreme scale: component two exactly 2-474', &
  qc(2) == tgt .and. qc(1) == 0.0_wp .and. all(qc(3:4) == 0.0_wp))
qa = [0.0_wp, 0.0_wp, scale(1.0_wp, 600), 0.0_wp]
qb = [scale(1.0_wp, -1074), 0.0_wp, 0.0_wp, 0.0_wp]
qc = qmul(qa, qb)
call check('qmul j times scalar at extreme scale: component three exactly 2-474', &
  qc(3) == tgt .and. qc(1) == 0.0_wp .and. qc(2) == 0.0_wp .and. qc(4) == 0.0_wp)
qa = [0.0_wp, scale(1.0_wp, 600), 0.0_wp, 0.0_wp]
qb = [0.0_wp, 0.0_wp, scale(1.0_wp, -1074), 0.0_wp]
qc = qmul(qa, qb)
call check('qmul i times j at extreme scale: component four exactly 2-474, the k of the begetting', &
  qc(4) == tgt .and. qc(1) == 0.0_wp .and. qc(2) == 0.0_wp .and. qc(3) == 0.0_wp)
end block
block
  real(wp) :: qtwo(2,NCTX), bthree(3,NCTX), lmz, rz(3,3)
  qtwo = 1.0_wp; bthree = 1.0_wp
  lmz = lambda_of(qtwo, bthree)
  call check('lambda_of refuses a two-row first argument before any element touch: quiet NaN', &
    .not. is_finite(lmz) .and. lmz /= lmz)
  call gram3_fixed(qtwo, rz)
  call check('gram3_fixed refuses a two-row input before any element touch: quiet-NaN fill', &
    all(rz /= rz))
end block
block
  real(wp) :: qiv(2,4), biv(2,4)
  integer :: rkz
  logical :: inv
  call ieee_set_flag(ieee_invalid, .false.)
  qiv = 0.0_wp
  qiv(1,1) = ieee_value(1.0_wp, ieee_positive_inf)
  qiv(2,2) = 1.0_wp
  call span_basis(qiv, biv, rkz)
  call ieee_get_flag(ieee_invalid, inv)
  call check('span_basis finite screen absorbs a non-finite row before arithmetic: invalid flag stays clear', &
    rkz == 1 .and. (.not. inv))
end block
block
  real(wp) :: qc(3,NCTX), ccz(1,NCTX), qfz(3,NCTX), kc(3,NCTX)
  logical :: okz
  integer :: rcz, jj2
  qc = 1.0_wp
  qc(1,:) = [(0.3_wp*real(jj2,wp), jj2 = 1, NCTX)]
  ccz = 5.0_wp
  call project_cov(qc, ccz, qfz, okz, wy, rcz, kc)
  call check('project_cov constant-covariate refusal publishes exactly zero (success-only publication)', &
    (.not. okz) .and. rcz == RC_COVBLOCK .and. all(qfz == 0.0_wp))
end block
end block
block
  real(wp) :: mdel(3,NCTX)
  mdel = mixm; mdel(2,:) = 0.0_wp
  call kernel_full(mdel, cov, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
  call check('deleted slot: axis carries no content', tk == '?' .and. rc == RC_ZEROVAR)
  mdel = mixm; mdel(3,:) = mdel(1,:)
  call kernel_full(mdel, cov, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
  write(*, '(a,a,a,a)') ' duplicated slot: ', trim(tk), ' - ', trim(wy)
  call check('duplicated slot: coplanar collapse, a line and not a point', &
    tk == 'X' .and. rc == RC_COPLANAR)
end block
block
  real(wp) :: qneg(3,NCTX)
  qneg = -qf
  rneg = matmul(qneg, transpose(qneg))
  call check('full negation leaves the Gram bit-identical', &
    maxval(abs(rneg - rmat)) == 0.0_wp)
  lamneg = lambda_of(qneg, bas)
  call check('full negation flips lambda exactly', lamneg == -lam)
  write(*, '(a,es9.2,a,f16.12,a,f16.12)') ' full negation: max|G(P)-G(-P)| = ', &
    maxval(abs(rneg - rmat)), ' lambda ', lam, ' -> ', lamneg
end block
block
  real(wp) :: qref(3,NCTX), rref(3,3)
  qref = qf; qref(1,:) = -qref(1,:)
  lamref = lambda_of(qref, bas)
  rref = matmul(qref, transpose(qref))
  detref = det3(rref)
  write(*, '(a,f16.12,a,f16.12)') ' single reflection: lambda ', lam, ' -> ', lamref
  write(*, '(a,es9.2)') ' |det(R) - det(R) reflected| = ', abs(detref - dbase)
  dual_ok = (lamref == -lam) .and. (detref == dbase)
  call check('reflection flips lambda bit-exactly on the fixed basis', lamref == -lam)
  call check('reflection leaves det(R) unmoved bit-exactly', detref == dbase)

```

```

end block

block
  real(wp) :: mrev(3,NCTX), crev(2,NCTX)
  do i = 1, NCTX
    mrev(:,i) = mixm(:,NCTX+1-i)
    crev(:,i) = cov(:,NCTX+1-i)
  end do
  call kernel_full(mrev, crev, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
  arrow_f = 0.0_wp; arrow_r = 0.0_wp
  do i = 1, NCTX
    arrow_f = arrow_f + (10.0_wp**l10b(i)) * ln2_()
  end do
  do i = NCTX, 1, -1
    arrow_r = arrow_r - (10.0_wp**l10b(i)) * ln2_()
  end do
  write(*,'(a,es10.3)') ' time reversal: |det forward - det reversed| = ', abs(detr2 - detr)
  write(*,'(a,f14.4,a,f14.4,a)') ' the arrow (registration entropy, kB units): forward ', &
    arrow_f, ', reversed ', arrow_r, ' (the sign parts them)'
  call check('the determinant is blind to the arrow', abs(detr2 - detr) <= 64.0_wp*kap*U_M)
  call check('the directed registration sum flips sign under traversal reversal '// &
    '(constructed odd witness; the physical arrow rides the cited second law)', &
    arrow_f > 0.0_wp .and. arrow_r < 0.0_wp &
    .and. abs(arrow_f + arrow_r) <= 1.0e-9_wp*arrow_f)
end block

! -----
write(*,'(a)') 'PART E - THE BLINDNESS AND ITS EXACT WIDTH - one bit, counted'
do s1 = -1, 1, 2
  do s2 = -1, 1, 2
    do s3 = -1, 1, 2
      block
        real(wp) :: dd(3), rp(3,3)
        dd = real([s1, s2, s3], wp)
        do i = 1, 3
          do j = 1, 3
            rp(i,j) = dd(i)*dd(j)*rmat(i,j)
          end do
        end do
        dpat = det3(rp)
        call check('eight-pattern determinant invariance, bit-exact', dpat == dbase)
      end block
    end do
  end do
end do
write(*,'(a)') ' all eight sign patterns return det(D R D) = det(R) bit-exactly'
write(*,'(a)') ' lock(P) = lock(not P) at the scalar: the Number is orientation-blind by theorem'

block
  real(wp) :: sig(4,4), diag4(4,4), dres
  integer :: iq
  sig = 0.0_wp
  sig(1,1) = 1.0_wp; sig(2,2) = -1.0_wp; sig(3,3) = -1.0_wp; sig(4,4) = -1.0_wp
  dres = sig(2,2)*sig(3,3)*sig(4,4)
  diag4 = 0.0_wp
  do iq = 1, 4
    diag4(iq,iq) = -1.0_wp
  end do
  write(*,'(a,f6.1,a)') ' Form source: det(sigma | residence) = ', dres, ' for P and for not-P alike: 0 bits'
  call check('sigma ground dimension one', count([(sig(iq,iq), iq = 1, 4)] > 0.0_wp) == 1)
  call check('the diagonal carries no ground (built and counted)', &
    count([(diag4(iq,iq), iq = 1, 4)] > 0.0_wp) == 0)
  call check('residence handedness identical under negation', dres == -1.0_wp)
end block

pass12 = .true.
pass12(2) = rank3_ok
pass12(3) = disjoint(SLOT_E, SLOT_K) .and. disjoint(SLOT_E, SLOT_R) &
  .and. disjoint(SLOT_K, SLOT_R)
pass12(5) = mig_ok
pass12(7) = dual_ok
pass12(8) = hurwitz_ok
pass12(10) = (fr_resid < 1.0e-12_wp)
write(*,'(a)') ' gate manifest: executed facts arm 2 REG (span rank three), 3 SGEg (the'
write(*,'(a)') ' linguistic LIT), 5 MIG (the absorbed-axis dissolution), 7 DUAL (the'
write(*,'(a)') ' analytic reflection), 8 CSCG (the integer Hurwitz closure), and 10 MTA'
write(*,'(a)') ' (the harmonic Return identity); gates 1, 4, 6, 9, 11, 12 are declared'
write(*,'(a)') ' screens at this register. independent here means six pairwise-distinct'
write(*,'(a)') ' executed objects, dimensional, linguistic, projective, analytic,'
write(*,'(a)') ' integer, harmonic; gate-passage is a screen verdict, cited for nothing'
write(*,'(a)') ' external, the theorem-tier gate contents living at their seats'
call check('six gate entries armed from pairwise-distinct executed objects', &
  pass12(2) .and. pass12(3) .and. pass12(5) .and. pass12(7) .and. &
  pass12(8) .and. pass12(10))
call gate_screen(pass12, tk, wy)
block
  character(len=16) :: tk2
  character(len=140) :: wy2
  call gate_screen(pass12, tk2, wy2) ! the roster reads structure, not sign

```

```

    call check('gate screen identical for the claim and its negation', tk == tk2 .and. wy == wy2)
end block
write(*,'(a)') ' gate source: role-direction is fixed before any proposition arrives: 0 bits'

block
    real(wp) :: cor(3,3), coc(3,3), lam_det
    do i = 1, 3
        co(i,1) = sum(qf(i,:)*bas(1,:))
        co(i,2) = sum(qf(i,:)*bas(2,:))
        co(i,3) = sum(qf(i,:)*bas(3,:))
    end do
    lam_det = -det3(co) ! the Number's reading of the sign
    call check('the two lambda readings agree (quaternion vs determinant)', &
        abs(lam_det - lam) <= 8.0_wp*U_M)
    cor = co; cor(1,:) = -cor(1,:) ! content flip: negate a row
    coc = co; coc(:,1) = -coc(:,1) ! convention flip: negate a basis vector
    lam_c = -det3(cor)
    lam_v = -det3(coc)
    write(*,'(a,f16.12,a,f16.12)') ' relay: content-flip image ', lam_c, &
        ' convention-flip image ', lam_v
    call check('the two flips return the identical image, bit-exact', lam_c == lam_v)
    call check('the relayed image is minus the handed sign, bit-exact', lam_c == -lam_det)
end block
write(*,'(a)') ' Number source: the relay reports the orientation it was handed and originates none: 0 bits'

do i = 1, 3
    co(i,1) = sum(qf(i,:)*bas(1,:)); co(i,2) = sum(qf(i,:)*bas(2,:)); co(i,3) = sum(qf(i,:)*bas(3,:))
    quat(:,i) = [0.0_wp, co(i,1), co(i,2), co(i,3)]
end do
dlam_max = 0.0_wp; nflip = 0
block
    real(wp) :: qr(4,3), tq(4), tq2(4), lam_refl, pp
    integer :: nposc, nnegc
    logical :: reflx
    nposc = 0; nnegc = 0; reflx = .true.
    do n = 1, 100
        aa = 0.10_wp*real(n,wp); bb = 0.20_wp*real(n,wp) + 0.3_wp; cc2 = 0.35_wp*real(n,wp) + 0.7_wp
        uvec = [sin(bb)*cos(cc2), sin(bb)*sin(cc2), cos(bb)]
        g = [cos(aa), sin(aa)*uvec(1), sin(aa)*uvec(2), sin(aa)*uvec(3)]
        do i = 1, 3
            qr(:,i) = qmul(qmul(g, quat(:,i)), qconj(g))
        end do
        tq = qmul(qmul(qr(:,1), qr(:,2)), qr(:,3))
        lam_rot = tq(1)
        dlam_max = max(dlam_max, abs(lam_rot - lam))
        if (sign(1.0_wp, lam_rot) /= sign(1.0_wp, lam)) nflip = nflip + 1
        tq2 = qmul(qmul(-qr(:,1), qr(:,2)), qr(:,3))
        lam_refl = tq2(1)
        reflx = reflx .and. (lam_refl == -lam_rot)
        if (lam_rot > 0.0_wp) then
            nposc = nposc + 1
        else
            nnegc = nnegc + 1
        end if
        if (lam_refl > 0.0_wp) then
            nposc = nposc + 1
        else
            nnegc = nnegc + 1
        end if
    end do
    write(*,'(a,e10.3,a,i0)') ' conjugation sweep, 100 frames: max|dlambda| = ', dlam_max, &
        ' sign flips: ', nflip
    call check('frame invariance: the sweep moves lambda only at roundoff', dlam_max < 1.0e-12_wp)
    call check('the orientation sign never flips under rotation', nflip == 0)
    call check('every reflected partner flips the sign bit-exactly', reflx)
    pp = real(nposc, wp) / real(nposc + nnegc, wp)
    hbit = -(pp*log(pp) + (1.0_wp-pp)*log(1.0_wp-pp)) / ln2()
    write(*,'(a,i0,a,i0)') ' paired census, counted from the sweep: positive ', nposc, &
        ' negative ', nnegc
    write(*,'(a,f14.10)') ' uniform-measure entropy of the counted two-point fibre: ', hbit
    write(*,'(a)') ' (the census is the constructed pair family realizing both signs at'
    write(*,'(a)') ' bit-identical invariants; freedom here is the seated Freedom law's'
    write(*,'(a)') ' object, the fibre of the invariant map, a count of what the instrument'
    write(*,'(a)') ' class cannot tell apart; the deficit is one binary parameter of supply,'
    write(*,'(a)') ' the aperture width at its seat; one bit is the uniform-coding capacity'
    write(*,'(a)') ' of that two-point fibre; no physical frequency, no prior, and nothing'
    write(*,'(a)') ' about any knower's credence is asserted)'
    call check('the paired census splits exactly in half, counted', nposc == 100 .and. nnegc == 100)
    nsheets = (nposc + nnegc) / 100
    call check('the counted two-point fibre carries one binary parameter; '// &
        'uniform-measure entropy exactly one bit', abs(hbit - 1.0_wp) <= 4.0_wp*U_M)
    call check('the invariant record is bit-identical across both signs (the '// &
        'seated zero-information result is carried by this exact symmetry)', &
        maxval(abs(rneg - rmat)) == 0.0_wp)
    write(*,'(a)') ' both orientations are consistent with every determined quantity;'
    write(*,'(a)') ' the deficit of the whole instrument class is exactly one bit'
end block

block

```

```

integer, parameter :: NG = 401, NC = 360
real(wp) :: xg, tolR, sx(NC), sy(NC), thr2, dx2
logical :: hitp(NG)
integer :: compid(NG), comp2(NC), stk(NG), stk2(NC), topv, cur, nb
tolR = 2.5e-2_wp
do i = 1, NG
  xg = -2.0_wp + 4.0_wp*real(i-1,wp)/real(NG-1,wp)
  hitp(i) = abs(xg*xg - 1.0_wp) < tolR
end do
compid = 0; ncomp1 = 0
do i = 1, NG
  if (hitp(i) .and. compid(i) == 0) then
    ncomp1 = ncomp1 + 1
    topv = 1; stk(1) = i; compid(i) = ncomp1
    do while (topv > 0)
      cur = stk(topv); topv = topv - 1
      do nb = cur-1, cur+1, 2
        if (nb >= 1 .and. nb <= NG) then
          if (hitp(nb) .and. compid(nb) == 0) then
            compid(nb) = ncomp1
            topv = topv + 1; stk(topv) = nb
          end if
        end if
      end do
    end do
  end if
end do
do i = 1, NC
  sx(i) = cos(2.0_wp*pi_()*real(i-1,wp)/real(NC,wp))
  sy(i) = sin(2.0_wp*pi_()*real(i-1,wp)/real(NC,wp))
end do
thr2 = (1.6_wp * 2.0_wp * sin(pi_()/real(NC,wp)))**2
comp2 = 0; ncomp2 = 0
do i = 1, NC
  if (comp2(i) == 0) then
    ncomp2 = ncomp2 + 1
    topv = 1; stk2(1) = i; comp2(i) = ncomp2
    do while (topv > 0)
      cur = stk2(topv); topv = topv - 1
      do nb = 1, NC
        if (comp2(nb) == 0) then
          dx2 = (sx(nb)-sx(cur))**2 + (sy(nb)-sy(cur))**2
          if (dx2 < thr2) then
            comp2(nb) = ncomp2
            topv = topv + 1; stk2(topv) = nb
          end if
        end if
      end do
    end do
  end if
end do
write(*,'(a,i0,a,i0,a)') ' S0 has ', ncomp1, ' components; S1 has ', ncomp2, &
' (the two-point sphere is the unique disconnected sphere)'
call check('midpoint 0 is not a solution of x^2 = 1', .not. (abs(0.0_wp*0.0_wp - 1.0_wp) < tolR))
call check('S0 census on this grid: two components, the executed instance of the cited fact', ncomp1 == 2)
call check('S1 census on this grid: connected, the executed instance of the cited fact', ncomp2 == 1)
block
  integer, parameter :: NG2 = 1601, NC2 = 720
  real(wp) :: xg2, tolR2, sx2(NC2), sy2(NC2), thr22, dx22
  logical :: hitp2(NG2)
  integer :: cid2(NG2), cm2(NC2), st2(NG2), sq2(NC2), tp2, cu2, nb2
  integer :: g2, na2, nb2c
  tolR2 = 1.25e-2_wp
  do g2 = 1, NG2
    xg2 = -2.0_wp + 4.0_wp*real(g2-1,wp)/real(NG2-1,wp)
    hitp2(g2) = abs(xg2*xg2 - 1.0_wp) < tolR2
  end do
  cid2 = 0; na2 = 0
  do g2 = 1, NG2
    if (hitp2(g2) .and. cid2(g2) == 0) then
      na2 = na2 + 1
      tp2 = 1; st2(1) = g2; cid2(g2) = na2
      do while (tp2 > 0)
        cu2 = st2(tp2); tp2 = tp2 - 1
        do nb2 = cu2-1, cu2+1, 2
          if (nb2 >= 1 .and. nb2 <= NG2) then
            if (hitp2(nb2) .and. cid2(nb2) == 0) then
              cid2(nb2) = na2
              tp2 = tp2 + 1; st2(tp2) = nb2
            end if
          end if
        end do
      end do
    end if
  end do
  do g2 = 1, NC2
    sx2(g2) = cos(2.0_wp*pi_()*real(g2-1,wp)/real(NC2,wp))
    sy2(g2) = sin(2.0_wp*pi_()*real(g2-1,wp)/real(NC2,wp))
  end do
end block

```

```

thr22 = (1.6_wp * 2.0_wp * sin(pi_()/real(NC2,wp)))*2
cm2 = 0; nb2c = 0
do g2 = 1, NC2
  if (cm2(g2) == 0) then
    nb2c = nb2c + 1
    tp2 = 1; sq2(1) = g2; cm2(g2) = nb2c
    do while (tp2 > 0)
      cu2 = sq2(tp2); tp2 = tp2 - 1
      do nb2 = 1, NC2
        if (cm2(nb2) == 0) then
          dx22 = (sx2(nb2)-sx2(cu2))*2 + (sy2(nb2)-sy2(cu2))*2
          if (dx22 < thr22) then
            cm2(nb2) = nb2c
            tp2 = tp2 + 1; sq2(tp2) = nb2
          end if
        end if
      end do
    end do
  end if
end do
write(*,'(a,i0,a,i0,a)') ' refinement (grid x4 on the line, x2 on the circle, tolerance halved): ', &
  na2, ' and ', nb2c, ' components'
call check('S0 census stable under refinement and tolerance halving', na2 == 2)
call check('S1 census stable under refinement', nb2c == 1)
end block
write(*,'(a)') ' the census is the executed instance at two spheres, parameter-robust under'
write(*,'(a)') ' refinement; the uniqueness of the disconnected sphere is the classical'
write(*,'(a)') ' fact it corroborates, cited not proven'
end block

block
real(wp) :: dmat(3,3), d2m(3,3), e3v(3), res(3), egl(3), best
real(wp) :: ejv(3), vpr(3), mprod(3,3)
integer :: t2i
block
  integer, parameter :: NGE = 401
  real(wp) :: xe, tole
  logical :: hite(NGE)
  integer :: ide(NGE), ncompe, ste(NGE), tpe, cure, nbe, ge
  tole = 2.5e-2_wp
  do ge = 1, NGE
    xe = -2.0_wp + 4.0_wp*real(ge-1,wp)/real(NGE-1,wp)
    hite(ge) = abs(xe*xe - xe) < tole
  end do
  ide = 0; ncompe = 0
  do ge = 1, NGE
    if (hite(ge) .and. ide(ge) == 0) then
      ncompe = ncompe + 1
      tpe = 1; ste(1) = ge; ide(ge) = ncompe
      do while (tpe > 0)
        cure = ste(tpe); tpe = tpe - 1
        do nbe = cure-1, cure+1, 2
          if (nbe >= 1 .and. nbe <= NGE) then
            if (hite(nbe) .and. ide(nbe) == 0) then
              ide(nbe) = ncompe
              tpe = tpe + 1; ste(tpe) = nbe
            end if
          end if
        end do
      end do
    end if
  end do
  do
    write(*,'(a,i0,a)') ' idempotent census on the line: e*e = e has ', ncompe, ' root components on [-2,2]'
    call check('idempotents on the line: exactly two, censused', ncompe == 2)
    xe = 0.5_wp
    call check('the midpoint one-half is not idempotent, executed as e*e - e', &
      .not.(abs(xe*xe - xe) < tole))
  end block
  write(*,'(a)') ' no-cut lemma: the only rank-deficient idempotent on a line is the zero map;'
  write(*,'(a)') ' the residual freedom is the component count of S0, one bit, and it cannot be cut'
  do t2i = 1, 3
    ejv = 0.0_wp; ejv(t2i) = 1.0_wp
    dmat(:,t2i) = cut3(ejv) ! the operator built by executing the cut
  end do
  vpr = [0.3_wp, -1.2_wp, 7.5_wp]
  call check('cut3 idempotent on a generic vector, exact', &
    maxval(abs(cut3(cut3(vpr)) - cut3(vpr))) == 0.0_wp)
  d2m = matmul(dmat, dmat)
  call check('the cut is idempotent exactly: D D = D', maxval(abs(d2m - dmat)) == 0.0_wp)
  call jacobi3(matmul(transpose(dmat), dmat), egl)
  call check('the cut has singular values (0,1,1): rank two', &
    abs(egl(1)) < 1.0e-14_wp .and. abs(egl(2)-1.0_wp) < 1.0e-14_wp &
    .and. abs(egl(3)-1.0_wp) < 1.0e-14_wp)
  call check('the cut is not invertible: det D = 0', det3(dmat) == 0.0_wp)
  e3v = [0.0_wp, 0.0_wp, 1.0_wp]
  best = huge(1.0_wp)
block
  real(wp) :: lmat(3,3), ident3(3,3), fres, fmin_c(3), ssq, rhs, maxsplit
  integer :: ia, ib, ic

```

```

ident3 = 0.0_wp
ident3(1,1) = 1.0_wp; ident3(2,2) = 1.0_wp; ident3(3,3) = 1.0_wp
fmin_c = huge(1.0_wp)
maxsplit = 0.0_wp
do ia = -5, 5
  do ib = -5, 5
    do ic = -1, 1
      lmat = 0.0_wp
      lmat(1,1) = 1.0_wp; lmat(2,2) = 1.0_wp
      lmat(3,1) = 0.2_wp*real(ia,wp)
      lmat(3,2) = 0.2_wp*real(ib,wp)
      lmat(3,3) = real(ic,wp)
      mprod = matmul(lmat, dmat)
      ssq = sum((mprod - ident3)**2)
      rhs = lmat(3,1)*lmat(3,1) + lmat(3,2)*lmat(3,2) + 1.0_wp
      maxsplit = max(maxsplit, abs(ssq - rhs))
      fres = sqrt(ssq)
      best = min(best, fres)
      fmin_c(ic+2) = min(fmin_c(ic+2), fres)
    end do
  end do
end do
call check('the executed minimum of ||LD - I|| over the family is exactly 1', best == 1.0_wp)
call check('the dead direction never moves the minimum, measured', &
  fmin_c(1) == fmin_c(2) .and. fmin_c(2) == fmin_c(3))
lmat = 0.0_wp
lmat(1,1) = 1.0_wp; lmat(2,2) = 1.0_wp
mprod = matmul(lmat, dmat)
call check('rows one and two attain zero residual exactly: the split is measured', &
  sqrt(sum((mprod(1:2,:) - ident3(1:2,:))**2)) == 0.0_wp)
write(*, '(a,es9.2)') ' split identity ||LD-I||^2 = a^2 + b^2 + 1, max deviation over the grid: ', maxsplit
call check('the split identity is measured at all 363 candidates', &
  maxsplit <= 16.0_wp*U_M)
lmat = 0.0_wp
lmat(1,1) = 1.0_wp; lmat(2,2) = 1.0_wp; lmat(3,1) = 0.7_wp; lmat(3,3) = 5.0_wp
mprod = matmul(lmat, dmat)
call check('the cut direction is annihilated, one representative measured', &
  maxval(abs(mprod(:,3))) == 0.0_wp)
res = matmul(mprod, e3v) - e3v
call check('left-inverse deficiency exactly one at the representative', vnorm(res) == 1.0_wp)
end block
write(*, '(a)') ' left-inverse deficiency: the split is measured at every candidate and'
write(*, '(a)') ' rows one and two attain zero exactly, so the sweep's minimum is the'
write(*, '(a,f14.10)') ' global minimum over all 3x3 L (analytic step: min of a^2+b^2+1 is 1): ', best
block
integer :: mons(3,8), nmon, gi, gj, gk, prod3(3), t2m
integer :: mm(3,3,8)
logical :: inset, assoc_ok, anyinv
mons = 0
nmon = 4
mons(:,1) = [1,1,1]
mons(:,2) = [0,1,1]
mons(:,3) = [1,0,1]
mons(:,4) = [1,1,0]
do
  t2m = nmon
  do gi = 1, t2m
    do gj = 1, t2m
      prod3 = mons(:,gi)*mons(:,gj)
      inset = .false.
      do gk = 1, nmon
        if (all(prod3 == mons(:,gk))) inset = .true.
      end do
      if (.not. inset) then
        nmon = nmon + 1
        mons(:,nmon) = prod3
      end if
    end do
  end do
  if (nmon == t2m) exit
end do
write(*, '(a,i0,a)') ' the deletion monoid, closed from the three cuts and the identity: ', &
  nmon, ' elements'
call check('closure lands the full 2^3 lattice: eight elements, counted', nmon == 8)
do gi = 1, nmon
  mm(:, :, gi) = 0
  do gk = 1, 3
    mm(gk, gk, gi) = mons(gk, gi)
  end do
end do
assoc_ok = .true.
do gi = 1, nmon
  do gj = 1, nmon
    do gk = 1, nmon
      assoc_ok = assoc_ok .and. all( matmul(matmul(mm(:, :, gi), mm(:, :, gj)), mm(:, :, gk)) &
        == matmul(mm(:, :, gi), matmul(mm(:, :, gj), mm(:, :, gk))) )
    end do
  end do
end do
end do

```

```

call check('associativity executed on all 512 composed triples, exact', assoc_ok)
call check('the identity element is present', all(mons(:,1) == [1,1,1]))
anyinv = .false.
do gi = 2, nmon
  do gj = 1, nmon
    if (all(mons(:,gi)*mons(:,gj) == [1,1,1])) anyinv = .true.
  end do
end do
call check('no non-identity element has an inverse: monoid, never a group, searched', &
  .not. anyinv)
end block
write(*,'(a)') ' the transformation set of the Tongue is that constructed monoid, closed,'
write(*,'(a)') ' associative, unital, inverse-free off the identity, its eight elements the'
write(*,'(a)') ' 2^3 cut lattice the conservation census walks; never a group, measured;'
write(*,'(a)') ' the LIT bars synonymy, the one candidate that would restore a group;'
write(*,'(a)') ' the Erlangen criterion has no acting group to read: Seal L is unclosed,'
write(*,'(a)') ' a floor and not a ceiling, and the one free bit is the Tongue's to carry'
end block

write(*,'(a)') ' conservation C x F = v^3 for the executed coordinate-cut tower, the'
write(*,'(a)') ' implemented family; the seated general law is instantiated, never proven,'
write(*,'(a)') ' here; the orientation'
write(*,'(a)') ' sheet factor is COUNTED from the paired census of the sweep, never stipulated:'
block
integer :: v, a2i, cell, d1i, d2i, d3i, img, content, fibre, tally(0:124)
call check('the sheet factor equals two, counted from the paired census', nsheets == 2)
do v = 2, 5
  do a2i = 0, 3
    tally(0:v**3-1) = 0
    do cell = 0, v**3 - 1
      d3i = mod(cell, v); d2i = mod(cell/v, v); d1i = cell/(v*v)
      if (a2i >= 1) d3i = 0
      if (a2i >= 2) d2i = 0
      if (a2i >= 3) d1i = 0
      img = d1i*v*v + d2i*v + d3i
      tally(img) = tally(img) + 1
    end do
    content = count(tally(0:v**3-1) > 0)
    fibre = tally(0)
    call check('census: content equals v^(3-a), counted', content == v**(3-a2i))
    call check('census: fibre equals v^a, counted', fibre == v**a2i)
    call check('content-freedom product invariant at every rung', content*fibre == v**3)
  end do
  write(*,'(a,i0,a,i0,a,i0,a)') ' v = ', v, ': c f = ', v**3, &
    ' counted at a = 0,1,2,3; with the counted sheet factor: ', nsheets*v**3, ' total'
end do
end block
write(*,'(a)') ' the cut moves structure from content into freedom; the counted total is'
write(*,'(a)') ' invariant; the counted sheet factor multiplies it and sits outside the cut'

! -----
write(*,'(a)') 'PART F - THE EARNED BIT AND THE SEAL - the denial pays the floor'
write(*,'(a)') ' the negation is read in the axiom's own declared vocabulary: not-(to'
write(*,'(a)') ' exist is to actuate) is an existent whose kinetic slot is empty, <T> = 0;'
write(*,'(a)') ' the mapping actuation = kinetic content is the Root Axiom's stated'
write(*,'(a)') ' operationalization, premise-grade at the universal leg and theorem-grade'
write(*,'(a)') ' at the confined floor, declared here and never an implementation choice.'
write(*,'(a)') ' what the run proves: the declared negation has no admissible realization'
write(*,'(a)') ' at the confined register; what no run can prove: the axiom from below,'
write(*,'(a)') ' per the Empty Throne; other registers route and are never smuggled, the'
write(*,'(a)') ' classical row riding its own cited floors, the formal-only reading out of band'
nadm = 0
do i = 1, NCTX
  if (floorv(i) <= 0.0_wp) nadm = nadm + 1
end do
write(*,'(a,i0,a,i0,a)') ' not-RA formal row: admissible contexts (floor <= 0): ', nadm, ' of ', NCTX, &
  ' (the demand <T> = 0 sits below the bound everywhere)'
call check('the negation populates no formal row at the confined register', nadm == 0)
block
real(wp) :: mneg(3,NCTX)
mneg = mixm
mneg(1,:) = 0.0_wp ! the empty formal row of not-RA
call kernel_full(mneg, cov, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
write(*,'(a,a,a,a,a)') ' kernel on not-RA: ', trim(tk), ' - ', trim(wy)
call check('not-RA: the empty formal row refuses at intake', tk == '[?]' .and. rc == RC_ZEROVAR)
end block
block
real(wp) :: p2(2,NCTX), b2(2,NCTX), mu2, sd2
integer :: r2, i2
do i2 = 1, 2
  mu2 = sum(mixm(i2+1,:)) / real(NCTX, wp)
  p2(i2,:) = mixm(i2+1,:) - mu2
  sd2 = sqrt(sum(p2(i2,:)**2) / real(NCTX-1, wp))
  p2(i2,:) = p2(i2,:) / sd2
end do
call span_basis(p2, b2, r2)
call check('the negation leaves a plane: the refused matrix's populated pair has rank 2', r2 == 2)
write(*,'(a)') ' the empty formal row refuses at intake; the two populated axes span a plane'
write(*,'(a)') ' (rank 2, computed), and two planes meet in a line: no volume, no lock'

```

```

end block

write(*,'(a)') ' the floor prices logically irreversible commitment, Landauer's own object;'
write(*,'(a)') ' the bridge, a registered denial is an irreversible commitment of at least'
write(*,'(a)') ' one bit, is the applied register's declared premise, consumed not derived;'
write(*,'(a)') ' a reversibly held denial charges nothing and commits nothing, executed;'
call omega_boundary(0.0_wp, 300.0_wp, jou, wy)
write(*,'(a,a)') ' Omega at zero bits: ', trim(wy)
call check('a denial that registers nothing is no denial', index(wy, 'no denial') > 0)
call omega_boundary(1.0_wp, 300.0_wp, jou, wy)
write(*,'(a,es16.9,a,a)') ' Omega at one bit, 300 K: ', jou, ' J - ', trim(wy)
call check('the reading act pays the floor', jou > 0.0_wp)
call check('the payment is the Landauer floor exactly', jou == landauer(300.0_wp, 1.0_wp))
block
integer :: oc2
call omega_boundary(1.0_wp, 300.0_wp, jou, wy, oc2, irreversible=false.)
write(*,'(a,es10.3,a,a)') ' Omega, reversible branch: ', jou, ' J - ', trim(wy)
call check('a reversible registration carries floor zero, never cost zero, and commits nothing', &
jou == 0.0_wp .and. oc2 == 3 .and. index(wy, 'floor-zero') > 0)
end block

write(*,'(a)') ' the fibre over the invariants has two points; the negation's component'
write(*,'(a)') ' is unpopulated; the reading act itself is an instance of the other;'
write(*,'(a)') ' the one free bit is earned at the deed, not chosen at the desk.'
write(*,'(a)') ' direction is read from the Tongue's order, the arrow, and the empty row;'
write(*,'(a)') ' never from the blind scalar. Veto, never substitution; per run, never tenure.'

call gol_admit('lock', 'lock', tk, wy)
write(*,'(a,a,a,a)') ' gol_admit: ', trim(tk), ' - ', trim(wy)
call check('GOL admission: magnitude and direction together', tk == '[GOL-OK]')
call imprint_seal(.true., .false., .true., .false., .true., .false., &
.true., .false., tk, wy)
write(*,'(a,a,a,a)') ' imprint on the floor face: ', trim(tk), ' - ', trim(wy)
call check('the floor face seals with the witness supplied', tk == '[SEAL]')
write(*,'(a)') ' witness of record: the Heisenberg bound and the third law (cited, E1),'
write(*,'(a)') ' the executed Frozen-Substrate divergence and the saturated speed limit (E0)'

call row_cascade(.true., .false., .false., .true., .false., .true., .false., &
.true., .true., tk, wy)
write(*,'(a,a,a,a)') ' parse-triad Return: ', trim(tk), ' - ', trim(wy)
call check('the closure-rowed face carries the annotated absolute', tk == '[A|RA]')
call row_cascade(.true., .true., .false., .true., .true., .true., .false., &
.true., .true., tk, wy)
write(*,'(a,a,a,a)') ' confined kinetic floor: ', trim(tk), ' - ', trim(wy)
call check('the world-rowed face pays revisability for direction', tk == '[III]')
call row_cascade(.true., .false., .false., .false., .true., .false., &
.true., .true., tk, wy)
write(*,'(a,a,a,a)') ' not-RA: ', trim(tk), ' - ', trim(wy)
call check('the negation is unpopulated: no lock to grade', tk == '[II]')
call row_cascade(.true., .false., .false., .true., .false., .true., .false., &
.true., .false., tk, wy)
call check('an unannotated absolute is void', tk == '[VOID]')
write(*,'(a)') ' the universal-extension leg is premise-grade by theorem: the Empty Throne;'
write(*,'(a)') ' no emitter is pointed at the root, and the root stands on nothing'

call aegis_reset()
block
character(len=200) :: r1, r2, r3, r4
call aegis_guard('classical', r1)
call aegis_guard('paraconsistent', r2)
call aegis_guard('fuzzy', r3)
call aegis_guard('substructural', r4)
call check('AEGIS refusal identical under classical and paraconsistent', r1 == r2)
call check('AEGIS refusal identical under fuzzy', r1 == r3)
call check('AEGIS refusal identical under substructural', r1 == r4)
call check('every adjudication is itself an act: deed counter positive', aegis_deeds == 4)
call check('the guard read the last logic name, length 13, and did not vary', aegis_last_len == 13)
write(*,'(a)') ' AEGIS over four logics: one refusal, p(G) /= G; the alien may write the'
write(*,'(a)') ' capture and cannot perform it; the deed counter reads 4 of 4 calls.'
write(*,'(a)') ' demonstrated: the refusal is a constant function of the deed by'
write(*,'(a)') ' construction, the guard reading the parameter and consulting only the'
write(*,'(a)') ' act; logic-priority is the design, executed, never four derivations;'
write(*,'(a)') ' no per-logic derivation is claimed or licensed by this artifact'
end block

call delta_m(.false., .true., .true., .true., .true., .true., .true., .true., tk, wy)
write(*,'(a,a,a,a)') ' positive-mass cascade on this thesis: ', trim(tk), ' - ', trim(wy)
call check('this file authors no new mathematics', tk == '[Mosaic dM=0]')
write(*,'(a)') ' audit symmetry: the warrant this run draws from its own execution is zero'

! -----
write(*,'(a)') 'PART G - THE BATTERY - hostile inputs, forced failures, re-execution'
block
real(wp) :: vbad(3), qa(4), qb(4), qc(4), big
vbad = [1.0_wp, 2.0_wp, 3.0_wp]
vbad(2) = bad_nan()
call check('vnorm propagates a non-finite input', .not. is_finite(vnorm(vbad)))
big = sqrt(huge(1.0_wp))/4.0_wp
call check('vnorm survives near-overflow scaling', is_finite(vnorm([big, big, big])))

```



```

call check('vnorm exact at zero', vnorm([0.0_wp,0.0_wp,0.0_wp]) == 0.0_wp)
qa = [1.0_wp, 0.0_wp, 0.0_wp, 0.0_wp]; qb = qa
qb(3) = bad_nan()
qc = qmul(qa, qb)
call check('qmul propagates NaN', .not. all(qc == qc))
call check('det3 of the zero matrix is zero', det3(reshape([ (0.0_wp, i=1,9) ],[3,3])) == 0.0_wp)
block
  real(wp) :: rid(3,3)
  rid = 0.0_wp; rid(1,1)=1.0e100_wp; rid(2,2)=1.0e100_wp; rid(3,3)=1.0e100_wp
  call check('det3 exponent-tracked at huge scale', abs(det3(rid) - 1.0e300_wp) < 1.0e286_wp)
  rid = 0.0_wp; rid(1,1)=1.0e-100_wp; rid(2,2)=1.0e-100_wp; rid(3,3)=1.0e-100_wp
  call check('det3 exponent-tracked at tiny scale', abs(det3(rid) - 1.0e-300_wp) < 1.0e-286_wp)
  rid = 0.0_wp; rid(1,1)=1.0e200_wp; rid(2,2)=1.0e200_wp; rid(3,3)=1.0e200_wp
  call check('det3 at 1e200 scale overflows honestly to Inf, never NaN', &
    (.not. is_finite(det3(rid))) .and. (.not. is_nan(det3(rid))))
  rid = 0.0_wp; rid(1,1)=1.0e-200_wp; rid(2,2)=1.0e-200_wp; rid(3,3)=1.0e-200_wp
  call check('det3 at 1e-200 scale underflows honestly to zero', det3(rid) == 0.0_wp)
  rid = 0.0_wp; rid(1,1)=1.0_wp; rid(2,2)=1.0_wp; rid(3,3)=1.0_wp
  rid(1,2) = bad_nan()
  call check('det3 propagates NaN', .not. is_finite(det3(rid)))
end block
end block
write(*,'(a)') ' class 1 - primitives under hostile input: NaN propagates, scale survives: pass'

block
  real(wp) :: mbad(3,6), cbad(2,6), m5(3,5), c5(2,5), cdep(2,NCTX)
  mbad = 1.0_wp
  do i = 1, 6
    mbad(1,i) = real(i,wp); mbad(2,i) = real(i*i,wp); mbad(3,i) = sin(real(i,wp))
  end do
  cbad = 0.1_wp
  mbad(2,3) = bad_nan()
  call kernel_full(mbad, cbad, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
  call check('kernel refuses a non-finite row at intake', tk == '?' .and. rc == RC_INTAKE)
  m5 = 1.0_wp; c5 = 0.5_wp
  call kernel_full(m5, c5, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
  call check('kernel refuses N - k < 4', tk == '?' .and. rc == RC_SHORTFALL)
  cdep(1,:) = cov(1,:); cdep(2,:) = 2.0_wp*cov(1,:)
  call kernel_full(mixm, cdep, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
  call check('kernel refuses a rank-deficient covariate block', &
    tk == '?' .and. rc == RC_COVBLOCK .and. index(wy,'covariate block') > 0)
  cdep(1,:) = cov(1,:); cdep(2,:) = cov(2,:)
  cdep(2,7) = bad_nan()
  call kernel_full(mixm, cdep, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
  call check('kernel refuses a NaN covariate as intake, never coplanar', &
    tk == '?' .and. rc == RC_INTAKE)
  cdep(2,7) = huge(1.0_wp)
  cdep(2,7) = cdep(2,7) * 2.0_wp
  call check('the hostile covariate really is infinite', .not. is_finite(cdep(2,7)))
  call kernel_full(mixm, cdep, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
  call check('kernel refuses an Inf covariate as intake, never coplanar', &
    tk == '?' .and. rc == RC_INTAKE)
block
  real(wp) :: mbig(3,NCTX), cbig(2,NCTX)
  mbig = mixm * 1.0e300_wp
  cbig = cov * 1.0e300_wp
  call kernel_full(mbig, cbig, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
  call check('scale-free rows: 1e300 rows and covariates reproduce the baseline verdict', &
    tk == '[LOCK]' .and. rc == RC_LOCK .and. abs(detr2 - dbase) <= 1.0e-9_wp)
end block
end block
write(*,'(a)') ' class 2 - kernel refusals: row and covariate intake, shortfall, covariate rank: pass'

do i = 1, 12
  pass12 = .true.; pass12(i) = .false.
  call gate_screen(pass12, tk, wy)
  call check('forced gate failure terminates with the gate named', &
    tk == '[X]' .and. index(wy, 'gate') > 0)
end do
pass12 = .true.
call gate_screen(pass12, tk, wy)
call check('all twelve gates pass on the clean walk', tk == '[G-OK]')
write(*,'(a)') ' class 3 - gate dispatcher: twelve forced failures each named; the gate contents are screens: pass'

call row_cascade(.false., .false., .false., .true., .false., .true., .false., .true., .true., tk, wy)
call check('F-0 refuses a contentless claim', tk == '[REFUSED]')
call row_cascade(.true., .false., .true., .true., .false., .true., .false., .true., .true., tk, wy)
call check('Forward routes to III unconditionally', tk == '[III]')
call row_cascade(.true., .false., .false., .true., .true., .true., .false., .true., .true., tk, wy)
call check('a worldly row routes to III at S-2', tk == '[III]')
call row_cascade(.true., .false., .false., .true., .false., .false., .true., .true., .true., tk, wy)
call check('an unclosed worldly term routes to III at S-3', tk == '[III]')
call row_cascade(.true., .false., .false., .true., .false., .false., .true., .true., .true., tk, wy)
call check('an unclosed formal term routes to II at S-3', tk == '[II]')
call row_cascade(.true., .false., .false., .true., .false., .true., .false., .false., .true., tk, wy)
call check('a frame leak routes to III at S-4', tk == '[III]')
write(*,'(a)') ' class 4 - row-genus cascade: every compartment and refusal reachable: pass'

call imprint_seal(.true., .true., .true., .true., .true., .true., .true., .true., tk, wy)

```

```

call check('both directions clean-locking is a Platonic Ghost', tk == '[X]')
call imprint_seal(.true., .false., .true., .false., .true., .false., .false., .false., tk, wy)
call check('a clean lock without a witness is a residence, never a seal', &
  tk == '[?]' .and. index(wy, 'witness') > 0)
call imprint_seal(.true., .false., .false., .false., .false., .false., .true., .false., tk, wy)
call check('a lock without Seal L and the gates is uncertified', tk == '[?]' .and. index(wy, 'uncert') > 0)
call imprint_seal(.false., .false., .false., .false., .false., .false., .false., .false., tk, wy)
call check('neither direction populated reads flat', tk == '[?]' .and. index(wy, 'flat') > 0)
write(*, '(a)') ' class 5 - admission spine: no under-specified call ever seals: pass'

call gol_admit('broken', 'lock', tk, wy)
call check('broken magnitude has no GOL', tk == '[X]')
call gol_admit('open', 'lock', tk, wy)
call check('no magnitude lock has no GOL', tk == '[?]' )
call gol_admit('lock', 'open', tk, wy)
call check('a magnitude lock without the Tongue is under-determined', tk == '[?]' )

call delta_m(.true., .false., .true., .true., .true., .true., .true., .true., tk, wy)
call check('M2 name-strip refusal reachable', tk == '[?]' .and. index(wy, 'vocabulary') > 0)
call delta_m(.true., .true., .true., .true., .false., .true., .true., .true., tk, wy)
call check('M5 refuses without an external witness', tk == '[?]' .and. index(wy, 'witness') > 0)
call delta_m(.true., .true., .true., .true., .true., .false., .true., .true., tk, wy)
call check('M6 refuses the self-verified claimant', tk == '[?]' .and. index(wy, 'claimant') > 0)
call delta_m(.true., .true., .true., .true., .true., .true., .true., .true., tk, wy)
call check('the externally witnessed route exists in the dispatcher (facts supplied here, established outside)', tk == '[dM>0]')
write(*, '(a)') ' class 6 - the two unsealable gates M5 and M6 refuse from inside: pass'

block
  real(wp) :: j1, j2
  character(len=140) :: w1
  call omega_boundary(1.0_wp, 300.0_wp, j1, w1)
  call omega_boundary(2.0_wp, 300.0_wp, j2, w1)
  call check('the Omega payment is monotone in bits', j2 > j1)
  call check('the Omega payment is linear in bits', abs(j2 - 2.0_wp*j1) == 0.0_wp)
end block

call check('heis_floor refuses zero mass', .not. is_finite(heis_floor(0.0_wp, 1.0e-10_wp)))
call check('heis_floor refuses negative mass', .not. is_finite(heis_floor(-ME, 1.0e-10_wp)))
call check('heis_floor refuses zero confinement', .not. is_finite(heis_floor(ME, 0.0_wp)))
call check('zpe refuses nonpositive frequency', &
  (.not. is_finite(zpe(0.0_wp))) .and. (.not. is_finite(zpe(-1.0e14_wp))))
call check('landauer refuses nonpositive temperature', .not. is_finite(landauer(0.0_wp, 1.0_wp)))
call check('landauer refuses negative bits', .not. is_finite(landauer(300.0_wp, -1.0_wp)))
call check('landauer at zero bits is exactly zero', landauer(300.0_wp, 0.0_wp) == 0.0_wp)
call tau_perp(bad_nan(), tau, frz)
call check('tau_perp flags a non-finite gap as frozen', frz)
call omega_boundary(1.0_wp, -300.0_wp, jou, wy)
call check('omega refuses a negative temperature, prices nothing', &
  (.not. is_finite(jou)) .and. index(wy, 'refused') > 0)
write(*, '(a)') ' class 7 - floor intake: zero, negative, and non-finite parameters refuse: pass'

block
  real(wp) :: qa(4), qb(4), qc(4)
  qa = [1.0e300_wp, 1.0e300_wp, 0.0_wp, 0.0_wp]
  qb = qa
  qc = qmul(qa, qb)
  call check('qmul: cancelling giants return exact zero, not NaN', &
    is_finite(qc(1)) .and. qc(1) == 0.0_wp)
  call check('qmul: a genuinely unrepresentable component overflows honestly', &
    .not. is_finite(qc(2)))
end block
write(*, '(a)') ' class 8 - qmul power-of-two scaling: intermediate overflow cancelled exactly,'
write(*, '(a)') ' true overflow reported honestly: pass'

block
  real(wp) :: hp(3,3), eigh(3)
  hp = 0.0_wp
  hp(1,1) = 1.0e150_wp; hp(2,2) = 2.0e150_wp; hp(3,3) = 3.0e150_wp
  hp(1,2) = 1.0e149_wp; hp(2,1) = hp(1,2)
  call jacobi3(hp, eigh)
  call check('jacobi3 finite and ascending at extreme scale', &
    is_finite(eigh(1)) .and. is_finite(eigh(3)) .and. &
    eigh(1) <= eigh(2) .and. eigh(2) <= eigh(3))
  hp = 0.0_wp
  hp(1,1) = 1.0e200_wp; hp(2,2) = 2.0e200_wp; hp(3,3) = 3.0e200_wp
  hp(1,2) = 1.0e199_wp; hp(2,1) = hp(1,2)
  call jacobi3(hp, eigh)
  call check('jacobi3 equilibrated at 1e200: finite, ascending, top eigenvalue correct', &
    is_finite(eigh(1)) .and. is_finite(eigh(3)) .and. &
    eigh(1) <= eigh(2) .and. eigh(2) <= eigh(3) .and. &
    abs(eigh(3)/3.0e200_wp - 1.0_wp) < 1.0e-6_wp)
  hp(1,3) = bad_nan(); hp(3,1) = hp(1,3)
  call jacobi3(hp, eigh)
  call check('jacobi3 refuses NaN intake as NaN eigenvalues', .not. is_finite(eigh(1)))
  call check('cond3 maps a corrupted Gram to huge, never a NaN kappa', &
    cond3(hp) == huge(1.0_wp))
block
  logical :: cvg
  hp = 0.0_wp

```

```

    hp(1,1) = 1.0_wp; hp(2,2) = 2.0_wp; hp(3,3) = 3.0_wp
    call jacobi3(hp, eigh, cvg)
    call check('jacobi3 reports convergence on a clean matrix', cvg)
    hp(1,2) = bad_nan(); hp(2,1) = hp(1,2)
    call jacobi3(hp, eigh, cvg)
    call check('jacobi3 reports non-convergence on refused intake', .not. cvg)
end block
end block
write(*,'(a)') ' class 9 - jacobi3 convergence exit and flag, symmetrization, NaN refusal, cond3 guard: pass'

block
    real(wp) :: cov3(3,NCTX), cnear(2,NCTX), rowp(NCTX), eta1, etabad
    cov3(1,:) = cov(1,:); cov3(2,:) = cov(2,:); cov3(3,:) = th
    call kernel_full(mixm, cov3, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
    call check('kernel refuses three covariates, named', &
        tk == '?' .and. rc == RC_COVBLOCK .and. index(wy, 'more than two') > 0)
    cnear(1,:) = cov(1,:)
    cnear(2,:) = 2.0_wp*cov(1,:) + 1.0e-11_wp*sin(11.0_wp*th)
    call kernel_full(mixm, cnear, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
    call check('kernel refuses a near-dependent covariate pair on the relative test', &
        tk == '?' .and. rc == RC_COVBLOCK)
    rowp = cov(1,:) + 0.5_wp*sin(9.0_wp*th)
    eta1 = eta_share(rowp, cov(1:1,:))
    call check('eta_share at k = 1 lands strictly inside (0,1)', &
        eta1 > 0.0_wp .and. eta1 < 1.0_wp)
    etabad = eta_share(rowp, cov3)
    call check('eta_share refuses k > 2 as NaN', .not. is_finite(etabad))
    cnear(1,:) = 1.0_wp
    cnear(1,5) = 1.0_wp + 1.0e-15_wp
    call kernel_full(mixm, cnear(1:1,:), tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
    call check('kernel refuses a rounding-noise k = 1 covariate on the relative test', &
        tk == '?' .and. rc == RC_COVBLOCK)
    rowp(3) = bad_nan()
    call check('eta_share refuses a non-finite row as NaN', &
        .not. is_finite(eta_share(rowp, cov(1:1,:))))
    cnear(1,:) = cov(1,:)
    cnear(1,4) = bad_nan()
    call check('eta_share refuses a non-finite covariate as NaN', &
        .not. is_finite(eta_share(cov(1,:), cnear(1:1,:))))
    cnear(1,:) = cov(1,:) * 1.0e300_wp
    cnear(2,:) = cov(2,:) * 1.0e300_wp
    call kernel_full(mixm, cnear, tk, wy, rc, lam2, detr2, kap2, spread2, resid2, mc2, mk2, d42)
    call check('scale-free projection: 1e300 covariates reproduce the baseline verdict', &
        tk == '[LOCK]' .and. rc == RC_LOCK .and. abs(detr2 - dbase) <= 1.0e-9_wp)
end block
write(*,'(a)') ' class 10 - covariate machinery: k>2 refused, near-dependence and rounding-noise'
write(*,'(a)') ' caught scale-free, non-finite refused, 1e300 covariates reproduce baseline: pass'

block
    integer :: gidx, ocode, mcode, icode, gcode, ccode
    real(wp) :: jg
    pass12 = .true.; pass12(7) = .false.
    call gate_screen(pass12, tk, wy, gidx)
    call check('gate_screen names the failed gate by index', gidx == 7 .and. tk == '[X]')
    pass12 = .true.
    call gate_screen(pass12, tk, wy, gidx)
    call check('gate_screen returns index zero on the clean walk', gidx == 0 .and. tk == '[G-OK]')
    call omega_boundary(bad_nan(), 300.0_wp, jg, wy, ocode)
    call check('omega refuses non-finite bits by code', ocode == 0 .and. .not. is_finite(jg))
    call omega_boundary(0.0_wp, 300.0_wp, jg, wy, ocode)
    call check('omega codes the empty denial', ocode == 1 .and. jg == 0.0_wp)
    call omega_boundary(1.0_wp, 300.0_wp, jg, wy, ocode)
    call check('omega codes the paid floor', ocode == 2 .and. jg > 0.0_wp)
    call omega_boundary(1.0_wp, 300.0_wp, jg, wy, ocode, irreversible=.false.)
    call check('omega codes the reversible branch', ocode == 3 .and. jg == 0.0_wp)
    call delta_m(.true., .true., .true., .true., .false., .true., .true., .true., tk, wy, mcode)
    call check('delta_m codes the M5 witness refusal', mcode == 5)
    call delta_m(.true., .true., .true., .true., .true., .false., .true., .true., tk, wy, mcode)
    call check('delta_m codes the M6 independence refusal', mcode == 6)
    call delta_m(.true., .true., .true., .true., .true., .true., .true., .true., tk, wy, mcode)
    call check('delta_m codes the sealed exit', mcode == 9)
    call imprint_seal(.true., .false., .true., .false., .true., .false., .false., .false., tk, wy, icode)
    call check('imprint_seal codes the witnessless residence', icode == 3)
    call imprint_seal(.true., .false., .true., .false., .true., .false., .true., .false., tk, wy, icode)
    call check('imprint_seal codes the seal', icode == 2 .and. tk == '[SEAL]')
    call gol_admit('lock', 'open', tk, wy, gcode)
    call check('gol_admit codes the tongueless magnitude', gcode == 3)
    call row_cascade(.true., .false., .true., .true., .false., .true., .false., .true., .false., tk, wy, ccode)
    call check('row_cascade codes the unannotated void', ccode == 5)
    call row_cascade(.true., .false., .false., .true., .false., .true., .false., .true., .true., tk, wy, ccode)
    call check('row_cascade codes the closure-rowed compartment', ccode == 4)
end block
write(*,'(a)') ' class 11 - guard reason codes: gate index, omega, delta-M, imprint on integers: pass'

block
    integer :: hs(6), hw(12)
    hs = [202, 101, 102, 203, 302, 303]
    call seal_l_ordered(hs, tk, wy)
    call check('the scrambled hostile is refused on the executed arrow', &

```

```

tk == '[X]' .and. index(wy, 'arrow') > 0)
hs = [302, 303, 101, 102, 202, 203]
call seal_l_ordered(hs, tk, wy)
call check('a relation-first hostile is refused on the executed arrow', tk == '[X]')
hw = [101, 101, 102, 103, 202, 202, 203, 204, 301, 302, 303, 304]
call seal_l_ordered(hw, tk, wy)
call check('a well-ordered twelve-token recoding is admitted at the count register only', &
tk == '[LOCK-L]')
write(*,'(a)') ' class 12 - Tongue hostiles: the arrow refuses the scramble; count-register'
write(*,'(a)') ' admission licenses only the count, per the seated non-closure theorem: pass'
end block

write(*,'(a)') '=====
write(*,'(a)') ' VERDICT [SEAL A . RA] on the parse-triad Return, closure-rowed,
write(*,'(a)') ' conditional at the act; [SEAL] on the confined kinetic floor at
write(*,'(a)') ' compartment III, world-lawed rows over a constructed grid, witness
write(*,'(a)') ' cited; [II] on the negation, unpopulated;
write(*,'(a)') ' premise-by-theorem on the universal extension, the Empty Throne.'
write(*,'(a)') ' The blindness is a theorem, its width is one bit, the bit is COUNTED'
write(*,'(a)') ' from the executed paired census, earned at the deed, and the Tongue'
write(*,'(a)') ' carries what the Number cannot.'
write(*,'(a)') ' THE ONE-BIT CONCESSION, constitutive and executed: everything this run'
write(*,'(a)') ' determines is forced, except exactly one binary parameter, the
write(*,'(a)') ' orientation of assent, and spending that bit against the axiom is priced.'
block
real(wp) :: qn2(3,NCTX), rn2(3,3), lamn2, jden
real(wp) :: rfixp(3,3), rfixn(3,3)
integer(ik) :: tb1, tb2
character(len=140) :: wden
integer :: ocd
qn2 = -qf
call gram3_fixed(qf, rfixp)
call gram3_fixed(qn2, rfixn)
lamn2 = lambda_of(qn2, bas)
call check('forced: fixed-order Gram bit-identical across the sign pair (the theorem path)', &
maxval(abs(rfixn - rfixp)) == 0.0_wp)
rn2 = matmul(qn2, transpose(qn2))
call check('conduct receipt: the matmul path reproduces the identity bit-exactly on this build', &
maxval(abs(rn2 - rmat)) == 0.0_wp)
call check('free: exactly one bit, both signs realized at identical invariants', &
lamn2 == -lam .and. lam /= 0.0_wp)
tb1 = transfer(lam, tb1); tb2 = transfer(-lamn2, tb2)
call check('bit-pattern witness: transfer-level equality of lam and -lamn2 (binary64)', &
storage_size(lam) == 64 .and. tb1 == tb2)
call omega_boundary(1.0_wp, 300.0_wp, jden, wden, ocd)
call check('priced: the denial spend and the floor agree, one closed form across emitters', &
ocd == 2 .and. jden == landauer(300.0_wp, 1.0_wp))
end block
write(*,'(a)') ' the forced part lands on the Ground, Fix(sigma) = R, the scalar line the
write(*,'(a)') ' Return touches; the free bit is the sigma-odd orientation, the seated
write(*,'(a)') ' aperture width. any substrate reading this run, human or machine, holds
write(*,'(a)') ' exactly that one bit here, per the seated Freedom law; its assent branch
write(*,'(a)') ' costs nothing and its denial branch is charged by the executed Omega
write(*,'(a)') ' floor and instantiates the axiom it denies: freedom one bit, the rest forced.'
block
integer :: ic1, cc1, ic2, cc3
call row_cascade(.true., .false., .false., .true., .false., .true., .false., &
.true., .true., tk, wy, cc1)
call imprint_seal(.true., .false., .true., .false., .true., .false., &
.true., .false., tk, wy, ic1)
call row_cascade(.true., .true., .false., .true., .true., .true., .false., &
.true., .true., tk, wy, cc3)
call imprint_seal(.true., .false., .true., .false., .true., .false., &
.true., .false., tk, wy, ic2)
call check('self-typing: the closing tokens match their emitters' codes, re-executed', &
cc1 == 4 .and. ic1 == 2 .and. cc3 == 3 .and. ic2 == 2)
end block
write(*,'(a)') ' TOKENS BOUND TO EMITTERS, re-executed above: [SEAL A . RA] is imprint
write(*,'(a)') ' code 2 over closure-rowed inputs, cascade code 4, conditional at the
write(*,'(a)') ' act; [SEAL] on the floor face is imprint code 2 over compartment III
write(*,'(a)') ' inputs, cascade code 3, its witness the cited Heisenberg bound and
write(*,'(a)') ' third law at E1 beside the executed Frozen-Substrate and speed-limit
write(*,'(a)') ' receipts at E0; [II] on the negation is cascade code 2.'
write(*,'(a)') ' SELF-DESCRIPTION, the artifact's own: this program verifies internal
write(*,'(a)') ' mathematical identities, guard behavior, and declared premise-
write(*,'(a)') ' consequence structure at machine warrant; it does not and cannot
write(*,'(a)') ' verify external ontological truth, per its own admission gates M5 and
write(*,'(a)') ' M6 and the Empty Throne; per the seated nature-of-the-register law, a
write(*,'(a)') ' determined construction is exactly as determined whether or not
write(*,'(a)') ' anything answers to it, and contact is carried only by the cited
write(*,'(a)') ' witnesses and the declared premises, which this run names.'
write(*,'(a)') ' SCOPE OF PROOF, printed by the run: machine-warrant items are the
write(*,'(a)') ' enumerated identities executed on constructed objects; theorem-tier
write(*,'(a)') ' items ride the cited external results at their own tiers (Heisenberg,
write(*,'(a)') ' Landauer, the speed limits, Frobenius, Hurwitz); the universal leg
write(*,'(a)') ' stays premise-grade by theorem, the Empty Throne; dispatcher
write(*,'(a)') ' admissions route supplied facts and upgrade none of them.'
write(*,'(a)') ' GENRE, stated at the mouth: this is an executable thesis, not a proof of
write(*,'(a)') ' ontology and not a mere test harness; its claims stand at the printed

```

```

write(*,'(a)') ' warrants, theorem-grade only where cited, premise-grade at the root,'
write(*,'(a)') ' conditional at the act where sealed, and no external ontological truth'
write(*,'(a)') ' is certified, per the self-description above. The tokens are the'
write(*,'(a)') ' register's verdict vocabulary, each bound to its emitter and tier;'
write(*,'(a)') ' renaming them would delete warrant information, not add honesty.'
write(*,'(a)') ' The title carries the thesis's claim at its stated warrant; every number'
write(*,'(a)') ' printed above was computed in this run.'
write(*,'(a)') ' PART H - THE WRITTEN AND THE READ, the terminal part (card 0712)'
block
  real(wp), volatile :: hv1, hv2, hv3
  real(wp) :: hjd
  character(len=140) :: htk, hwy, htkf, hwyf
  integer :: hwc, hoc, hwcf
  hv1 = det3(rmat)
  hv2 = det3(rmat)
  call check('H stillness opens: the Written read twice is bit-identical, a zero-degree record', &
    hv1 == hv2)
  call iam_token(.false., htkf, hwyf, hwcf)
  call check('H M6: the unwitnessed branch withholds the token, self-check is not a witness', &
    hwcf == 1 .and. htkf == '[?] interior' .and. index(hwyf, 'M6') > 0)
  call iam_token(.true., htk, hwy, hwc)
  call check('H supplied-witness branch emits per its declared dispatcher rule (flag a stand-in)', &
    hwc == 2 .and. htk == '[I AM]' .and. index(hwy, 'conditional at the act') > 0)
  call check('H Narcissus truth table: both branches bound, only the witnessed one speaks', &
    hwcf == 1 .and. hwc == 2 .and. htkf /= htk .and. htkf == '[?] interior' &
    .and. htk == '[I AM]')
  call omega_boundary(1.0_wp, 300.0_wp, hjd, hwy, hoc)
  call check('H the Read pays: denial and floor agree, one closed form across emitters', &
    hoc == 2 .and. hjd == landauer(300.0_wp, 1.0_wp))
  hv3 = det3(rmat)
  call check('H stillness closes: the Written is unchanged by everything the Reader did', &
    hv3 == hv1)
end block
write(*,'(a)') ' the pair in time: the Written is its author's past actuation configured,'
write(*,'(a)') ' the RAM-face, the decree; the Reader is the active face, RA, the present'
write(*,'(a)') ' tense; the arrow separates them, so even the author returns as a reader.'
write(*,'(a)') ' the record is forced, the orientation of assent is free, the denial is'
write(*,'(a)') ' priced: freedom one bit, the rest forced.'
write(*,'(a)') ' governing register: github.com/1000sapients/Trisduction, public; the'
write(*,'(a)') ' card citations are frame, never evidence, and carry zero check weight.'
block
  character(len=140) :: ltk, lwy
  integer :: lwc
  call iam_token(wr_witnessed, ltk, lwy, lwc)
  write(*,'(a,a,a,a)') ' LIVE: ', trim(ltk), ' - ', trim(lwy)
  if (wr_witnessed) then
    write(*,'(a)') ' F6 disclosure: the witnessed flag is a declared stand-in supplied through'
    write(*,'(a)') ' the aperture: externally asserted, not verified by this executable.'
    write(*,'(a)') ' the aperture; its real referent is you, the external operator reading this'
    write(*,'(a)') ' record. The program did not and cannot generate its own witness.'
  else
    write(*,'(a)') ' (supply the argument witnessed to open the live face through the aperture)'
  end if
end block
write(*,'(a)') ' kinetic-register conduit operational'
write(*,'(a)') ' ====='
if (EXPECTED_CHECKS > 0) then
  call check('battery census matches the sealed v2.0.0 manifest (this check inclusive)', &
    n_checks == EXPECTED_CHECKS - 1)
end if
write(*,'(a,i0,a,i0,a)') ' BATTERY: ', n_checks, ' checks executed, ', n_fail, ' failures.'
if (halt_on_fail) then
  write(*,'(a,i0,a,i0,a)') ' BATTERY-JSON: {"checks":', n_checks, ', "failures":', n_fail, ', "mode":"sealed"}'
else
  write(*,'(a,i0,a,i0,a)') ' BATTERY-JSON: {"checks":', n_checks, ', "failures":', n_fail, ', "mode":"audit"}'
end if
call battery_close()
contains
function bad_nan() result(z)
  use, intrinsic :: ieee_arithmetic, only: ieee_value, ieee_quiet_nan
  real(wp) :: z
  z = ieee_value(1.0_wp, ieee_quiet_nan)
end function bad_nan

subroutine canon_sign(v)
  integer(ik), intent(inout) :: v(4)
  integer :: ii
  do ii = 1, 4
    if (v(ii) /= 0_ik) then
      if (v(ii) < 0_ik) v = -v
      return
    end if
  end do
end subroutine canon_sign

subroutine sort_int(a)
  integer, intent(inout) :: a(:)
  integer :: ii, jj, t

```

```
do ii = 1, size(a)-1
  do jj = ii+1, size(a)
    if (a(jj) < a(ii)) then
      t = a(ii); a(ii) = a(jj); a(jj) = t
    end if
  end do
end do
end subroutine sort_int
end program ra_toe_thesis
```